State of Private Transfers Report 2026

An in-depth analysis of private transfers protocols in the Ethereum ecosystem

Privacy Stewards of Ethereum

Private Transfers Engineering

Introduction

As blockchain networks have scaled and matured, the need for strong privacy has become a clear requirement. Individuals cannot have self-sovereignty without privacy, and institutions need privacy in place to deploy onchain. The transfer of money is the most fundamental aspect of blockchain privacy, and is the focus of this report.

The following report conducts an in-depth analysis of a variety of Ethereum protocols. We analyse 13 privacy protocols across 31 properties. We outline the properties we analyse protocols against, and then review each protocol against each property. Each protocol can be compared in the table of evaluations. We conclude on the state of the ecosystem and outline future work.

There are an abundance of privacy protocols on Ethereum which employ a large variety of different technologies and approaches. This report is the first snapshot of this ecosystem — for an up-to-date and evolving view on this space, you can find the latest updates on our dashboard: https://private-transfers.pse.dev/.

What is a Private Transfer?

We will use the following definitions for defining a private transfer:

- **Pseudonymity** - the status-quo on Ethereum today. Accounts have addresses that can be tied to real-world identities
- **Confidentiality**: Balances and amounts are private
- **Anonymity**: Sender and receiver are private
- **Asset Privacy**: The asset being transferred is private
- **Full privacy**: having confidentiality, anonymity, asset privacy at the same time

Some other ways of thinking about privacy. There is another way to classify privacy definitions which might be helpful for the reader:

- **Unlinkability**: Given two accounts A and B belonging to the same user, an observer cannot determine that A and B are related.
- **Untraceability**: Given a payment from Alice to Bob's account, an observer cannot identify Bob as the recipient.
- **Balance Hiding**: An observer cannot determine the total balance held across a user's stealth addresses.
- **Pattern Hiding**: An observer cannot build behavioral profiles from fragmented transaction histories.

Categories

Technology categories

A number of technologies can and are used amongst private transfer protocols.

- Zero Knowledge Proofs (ZKPs)
- Fully homomorphic encryption (FHE)
- Homomorphic encryption (HE)
- Multi Party Computation (MPC)
- TEEs
- Garbled Circuits
- coSNARKs
- Post-quantum cryptography

Architecture categories

- **Mixer** - a service that allows you to mix funds by transferring funds into a pool, and then later withdraw those funds without linking them to your previous address. No transfer functionality
- **Shielded pool** - a smart contract pool where assets sit as encrypted UTXO notes inside a Merkle tree. Transfers happen by spending and creating notes inside the pool.
- **Stealth addresses** - A scheme where each payment goes to a fresh one-time address derived from the recipient's public spending and viewing keys. Outside observers can't link payments to a single account. The recipient scans the chain with their viewing key to find and spend funds
- **Encrypted Tokens** - tokens whose balances and transfer amounts stay encrypted on-chain. Computations can be made on encrypted state without revealing values
- **Private L2** - a L2 rollup with privacy features. Inherits data availability and settlement security from Ethereum while keeping transaction contents shielded.
- **Private Plasma** - TODO
- **Private Validium** - A Validium is a scaling solution that enforces integrity of transactions using validity proofs like ZK rollups, but doesn’t store transaction data on the Ethereum itself
- **zkWormholes (burn-and-mint)** - Send funds to an unclaimable address and use zero-knowledge proofs to prove you own those funds in order to mint new tokens. The recipient mints to any address by privately proving the source is a valid burn, thus breaking the identifiable link between burn and mint
- **Decentralised Network** - a standalone decentralised network focussing on providing an additional service. Sometimes with additional crypto-economic assumptions. For example, a

FHE or TEE coprocessor.

- **Privacy stack/layer/middleware** - a full stack privacy solution which incorporates multiple technologies and components. Or a privacy layer that sits between users and the chain to provide privacy.
- **Bytecode Obfuscator** - a technique that deterministically produces many functionally equivalent variants of the same contract logic. Each private transfer can settle through a freshly-deployed escrow contract that looks like unrelated bytecode, so transfers don't share an on-chain anchor that observers can cluster against
- **Private VPN** - a privacy layer that sits between the user's wallet and the chain, as an RPC proxy. The user signs a regular transaction and the proxy generates a zero-knowledge proof that the transaction and its ECDSA signature are valid - this proof and the hidden transaction details can be used to interact with a privacy protocol.
- **Cross-chain swap aggregator** - routes value across L1s via exchanges or other off-chain hops to break on-chain linkability between source and destination addresses

Properties Definition

Privacy

- **Anonymity** set size: Number of entities participating in the protocol TODO: drop

1. **Confidentiality**: Balances and amounts are private
2. **Anonymity**: Sender and receiver are private
3. **Asset privacy**: The asset being transferred is private
4. **Plausible deniability**: Whether it is possible to detect if you are interacting with the privacy protocol. transfers are indistinguishable from public transfers

Cost and Performance

TODO: drop cost and performance as it’s in the dashboard and won’t be applied to every project

- **On-chain gas cost: deposit**: The gas cost of submitting a deposit transaction on-chain
- **On-chain gas cost: transfer**: The gas cost of submitting a transfer transaction on-chain
- **On-chain gas cost: withdraw**: The gas cost of submitting a withdraw transaction on-chain

UX

6. **Number of secrets**: How many secrets does the app or protocol require a user to store?

7. **Time-to-finality**: The time it takes for a transaction is considered irreversible and permanently part of the blockchain
8. **Deposit time**: Duration required before you can deposit into the protocol
9. **Withdraw time**: Duration required before you can withdraw funds from the protocol

Decentralization & Security

10. **Censorship resistance**: Ability to use the protocol without any restriction or being censored
11. **External network dependence**: Whether the protocol relies on an external network with additional crypto-economic assumptions
12. **Escape hatch**: Whether users can withdraw funds relying only on Ethereum consensus, smart contracts, and cryptography
13. **Upgradeability**: How upgrades to the system are performed
14. **Client-side proving**: Whether the protocol generates proofs on the client device or if it depends on an external service
15. **Third-party inspectability**: Whether a third party or parties can inspect the private info of the users. This can happen if the protocol uses an external prover to generate proofs.
16. **Implementation maturity**: How developed and battle-tested the protocol is, measured by type of deployment, production readiness, and amount of time deployed on mainnet. Maturity weights follow a 1–5 scale.
17. **Post-quantum secure**: Is the protocol secure against quantum threats

Compliance

18. **Layer of enforcement**: Where is compliance enforced in the protocol? Is it enforced on the blockchain itself? The protocol/app being used, or the underlying asset?
19. **Enforcement entities**: Which entity or entities enforce compliance restrictions
20. **Type of compliance**: What type of compliance is being enforced
21. **Point of enforcement**: Where the compliance is enforced in the private transfer lifecycle. When depositing, transferring within the protocol, or withdrawing.
22. **Selective disclosure**: viewing entity: The ability to share only what's needed (e.g. proving you own an NFT without revealing your entire wallet history)
23. **Selective disclosure**: Viewing control: How selective disclosure viewing control managed

Verifiable

24. **Cryptographic verifiability**: Whether correctness is guaranteed by cryptography rather than economic and/or majority-based mechanisms

25. **Open source**: The source code for the underlying protocol, any backend infrastructure, and any frontend applications is publicly available to inspect and has an open source software license.

State

26. **Private State Scalability**: How protocol-specific private data is stored over time
27. **Client-side indexing**: Whether user devices must continuously scan the chain to track balances or accounts
28. **Private state model**: What model does the protocol use for managing private state
29. **On-chain storage**: Where is the submitted on-chain data stored

Composability

30. **Access to DeFi**: Whether the solution can be used directly with DeFi applications. Whether the solution provides a native DeFi ecosystem for shielded assets Or whether there is no access to DeFi
31. **Programmability/Generality**: Range and expressiveness of private logic: from simple payment flows to rich private smart-contract ecosystems.

Evaluations

We evaluated 13 privacy protocols that enable private transfers. They are ordered alphabetically.

Bermuda

Bermuda is a privacy protocol built for EVM chains that acts as a privacy layer between applications and the underlying chain. The protocol gives existing EVM applications shielded accounts, stealth-address payments, with built-in compliance features without requiring smart contract changes. It combines zero-knowledge proofs with programmable compliance (deposit screening, retroactive flagging, withdrawal proofs). Bermuda supports private, compliant, and gasless transactions and is advertised as enterprise grade

Categories: Shielded pool, Stealth addresses, Zero Knowledge Proofs (ZKPs), Privacy Stack/Layer/Middleware

Documentation: https://docs.bermudabay.xyz

Confidentiality — Yes. Shielded balances and transfer amounts are held inside encrypted note commitments. The shielded pool holds all shielded funds and manages deposits, transfers, and withdrawals by verifying ZK proofs. Every UTXO is encrypted with X25519-XChaCha20-Poly1305 — note values and the resulting user balances are hidden to outside observers.

Anonymity — Yes. The shielded pool architecture means that private transfers do not reveal the sender or recipient. Like many other privacy protocols, operations are run through a relayer that submits shielded transactions on behalf of users, decoupling the sender's account from the on-chain transaction. Withdrawal proofs let the pool verify the withdrawal on-chain without revealing the user's identity, balance, or full transaction history. Proof generation and spending keys stay client-side, transaction details never leave the user's device.

Asset privacy — Yes. Bermuda routes deposits, transfers, and withdrawals through a single shielded pool contract that holds multiple ERC-20 tokens.

Plausible deniability — No. Bermuda transactions are distinguishable from ordinary transfers because operations route through protocol-specific contracts. The shielded pool is the core smart contract that holds all shielded funds and manages deposits, transfers, and withdrawals, so any deposit is an observable call to a known privacy contract.

Time-to-finality — N/A. Bermuda is an application-layer protocol deployed on host EVM chains, so transaction finality is inherited from the underlying network rather than imposed by Bermuda itself.

Number of secrets — 1. Each Bermuda account is controlled by a spending and an encryption key pair, both deterministically derived from a single random seed. Using the user's wallet private key as seed inherits the host wallet's recovery setup, so no new secret beyond the existing wallet key is required. It is also possible to generate keys by using a signature over a fixed message as a seed. This path is not recommended because signature-based key generation introduces phishing attack vectors.

Deposit time — 0. Deposits are ordinary on-chain transactions: the SDK exposes a single deposit call that funds a Bermuda account with any ERC-20 token, with no queue, challenge window, or time-lock beyond host-chain block inclusion. Pre-shield screening runs through Predicate as part of the same transaction, so attestation latency does not delay the deposit itself.

Withdraw time — 0. Withdrawals are single on-chain transactions: the SDK exposes a withdraw call that sends any shielded ERC-20 token to a public Ethereum account, gated only by ZK proof verification with no queue, exit window, or time-lock beyond host-chain block inclusion. Blacklist root checks run inline with the withdrawal proof and do not add a protocol-imposed wait.

Censorship resistance — No. Bermuda enforces compliance through two on-chain gates that can block valid transfers. At deposit, the depositor address is screened against the active policy through a Predicate-based attestation flow, and a failing address has its deposit rejected before funds enter the pool. At withdrawal, the pool checks the withdrawal against the current blacklist root maintained by the compliance gateway, forcing blacklisted funds to become public and removing the private exit.

External network dependence — Yes, permissioned. Bermuda depends on Predicate, an external attestation service, for pre-shield screening. Without a valid Predicate-backed attestation, deposits are rejected before funds enter the pool.

Escape hatch — Instantly. Bermuda's pool and verifier contracts are immutable, and withdrawals are not blocked by compliance. If a deposit lineage is retroactively flagged, the user can still withdraw — funds exit but they lose privacy.

Upgradeability — Immutable. The pool and its associated verifier contracts are fully immutable, and the pool itself is a standard Solidity smart contract without any privileged admin functions, so no central party can alter pool or verifier behaviour after deployment. The compliance gateway blacklist root is updated by an off-chain compliance engine each time the curated list changes, making that one part of state mutable.

Client-side proving — Yes. Proof generation runs entirely on the user's device. All ZK proof generation happens client-side inside the prover. Spending keys and transaction details never leave the user's device. The circuits are written in Noir with the Barretenberg backend, and the design targets universal client-side proving on standard consumer hardware, with further proving-pipeline optimisations under evaluation.

Third-party inspectability — No. Proof generation runs locally, so ZK proof generation happens client-side inside the embedded prover, and spending keys and transaction details never leave the user's device. The relayer and bundler dispatch transactions but, by design, are off-chain services that facilitate on-chain execution without having access to private transaction data. Withdrawal verification preserves confidentiality as well, since the pool can verify the withdrawal on-chain without revealing the user's identity, balance, or full transaction history.

Implementation maturity — 2 : Public testnet. The Bermuda wallet package is in beta with Plasma testnet as the only supported network, and the SDK is available to partners only by direct outreach. Bermuda is not deployed to mainnet.

Post-quantum secure — No. Spending key pairs produce Schnorr signatures and operate over the Grumpkin curve, an elliptic-curve construction whose discrete-log hardness is broken by Shor's algorithm. UTXO encryption uses X25519-XChaCha20-Poly1305, ECDH over Curve25519 paired with XChaCha20-Poly1305, and the ECDH component is likewise vulnerable to a cryptographically relevant quantum adversary.

Layer of enforcement — Protocol/chain. Compliance is enforced at the protocol's own contract layer through the on-chain compliance gateway, which governs fund flow by checking deposits against the blacklist and verifying that withdrawals meet configured policies. The pool contract itself manages deposits, transfers, and withdrawals by verifying ZK proofs and enforcing compliance checks. Policies are configurable per integrator.

Enforcement entities — Third party. Bermuda's compliance decisions are sourced externally: deposit screening runs through a Predicate attestation flow that checks the request against the active policy, and retroactive blacklisting is driven by external intelligence feeds such as sanctions lists (OFAC, EU, UN), law enforcement investigations, and on-chain forensic analysis. The compliance engine maps flagged addresses to deposit_id lineage and publishes a blacklist root, but the dispositive verdict originates from the external attestor and listed sources. Integrator configurability is limited to picking which feeds and update frequencies to consume.

Type of compliance — Proof of innocence (POI) / ASP, Programmatic policies. Bermuda enforces compliance through two mechanisms. At deposit, depositor addresses are screened against the active policy through a Predicate attestation flow, and failing addresses are rejected. At withdrawal, a private proof of innocence proves via zero-knowledge that the withdrawn funds exclude blacklisted deposit_id lineage under the current blacklist state, with the client proving exclusion against the blacklist root and the pool verifying that proof on-chain. The curated blacklist maps flagged addresses to deposit_id values, with an updated root published for withdrawal proofs.

Point of enforcement — Deposit, Withdrawal. Compliance is enforced at deposit and withdrawal. Before public assets enter the pool, the depositing address is screened and the deposit is authorized through attestations together with deposit-time compliance data. At exit, the withdrawal is checked against the current blacklist state, with compliant funds retaining a private withdrawal path through a proof of innocence while blacklisted lineage is forced onto the public path. Retroactive flagging operates between these two endpoints but ultimately fires at the withdrawal gate.

Selective disclosure: viewing entity — User, Voluntary third-party disclosure. A Bermuda account is a shielded account with a spending key pair and an encryption key pair, both held by the user. The pool verifies withdrawals on-chain without revealing the user's identity, balance, or full transaction history. A withdrawal proof can also be shared voluntarily with a third party — documented counterparties include centralized exchanges accepting withdrawals without additional KYC, recipients verifying untainted funds, and regulators auditing the withdrawal check without accessing user data. The third-party path is opt-in and triggered by the user sharing the proof, fitting voluntary disclosure rather than an involuntary or protocol-mandated viewing channel.

Selective disclosure: viewing control — Pre-defined. Disclosure happens through a per-withdrawal proof of innocence that the user may share with a counterparty (exchange, recipient, regulator) to verify the withdrawal check. The proof is bound to that single withdrawal at proving time, so the disclosure scope is fixed when the proof is issued — the user controls whether to share, but the granularity is per-withdrawal.

Cryptographic verifiability — Yes. Transaction correctness on Bermuda rests on zk-SNARK verification performed by on-chain contracts. The pool contract holds shielded funds and manages deposits, transfers, and withdrawals by verifying ZK proofs and enforcing compliance

checks. Circuits are written in Noir and use the Barretenberg backend. Verifier contracts perform ZK-SNARK proof verification, with multiple verifier instances covering different UTXO topologies and spending scenarios, so verifier selection is per-circuit rather than majority-based.

Open source — No. The wallet SDK repository wdk-wallet-bermuda is published under Apache License 2.0, an opem source licence. The public organisation listing shows Apache-2.0 on wdk-wallet-bermuda and the schnorr fork, with other repositories under GPL-3.0, MIT, or no declared licence. The circuits and contracts are not publicly inspectable from the github organisation.

Private State Scalability — Infinity grow. Bermuda's shielded pool stores encrypted UTXOs on-chain, and the pool contract holds all shielded funds across deposits, transfers, and withdrawals. Each spend adds new commitments and consumes nullifiers held by the pool.

Client-side indexing — No scanning. A separate chain-state service periodically downloads the most recent chain state via scheduled GitHub Actions, producing pre-indexed snapshots that the SDK ingests. The client SDK runs a single bootstrap synchronization step that loads this snapshot before performing ZK proving, rather than scanning every block itself. The indexing work is performed by the chain-state crawler service.

Private state model — UTXO-based state model. Bermuda uses a UTXO-based shielded pool: state is organised as discrete encrypted outputs rather than per-account ciphertext balances. Each spend produces new commitments and consumes nullifiers held by the pool contract.

Private Data Storage — Smart contracts. The shielded pool contract is the authoritative store. The on-chain pool holds all shielded funds and manages deposits, transfers, and withdrawals by verifying ZK proofs and enforcing compliance checks. Encrypted UTXO ciphertexts are the unit of private state and are emitted through the contract layer so recipients can trial-decrypt them. UTXOs use ephemeral Curve25519 key pairs with the public key placed in the authenticated plaintext portion of the AEAD envelope, eliminating the need for any public key registries — any off-chain index is a derived mirror of contract history rather than authoritative storage.

Access to DeFi — Unlimited access to DeFi applications. Bermuda routes external DeFi calls through burner EIP-7702 smart accounts that can target any external EVM smart contract. The SDK documents both stateless interactions (swapping on Uniswap) and stateful ones (supplying or withdrawing liquidity to an Aave pool), with ERC-4626 tokenized vaults supported out-of-the-box by passing the vault's share token address as the token parameter. The burner-account pattern is generic across external contracts rather than a curated allowlist, so composition is mediated by the unshield/reshield flow but the destination set is open.

Programmability / Generality — Transfers and DeFi operations. Bermuda's shielded layer exposes deposit, transfer, and withdraw operations over ERC-20 tokens, with no native private smart-contract execution environment. For richer logic, interactions with DeFi protocols utilizing shielded funds are routed through burner EIP-7702 smart accounts that call external contracts, supporting both stateless interactions like swapping on Uniswap and stateful ones like supplying

or withdrawing liquidity from an Aave pool. ERC-4626 vault shares are handled as ordinary shielded ERC-20s. Programmability is thus bounded to payments plus unshield/reshield-wrapped calls into external public protocols, not arbitrary private state.

Curvy

Curvy is a privacy-preserving cross-chain payment protocol with compliance features. It combines stealth addressses with ZK-SNARKs

Categories: Stealth addresses

Documentation: https://docs.curvy.box/

Confidentiality — Yes. Amounts and balances are private only in specific flows. When splitting, aggregating, or performing a private transfer between two users inside the privacy aggregators, all qualities of the transaction — including amount and currency — remain completely private. However, shielding funds exposes the amount and currency publicly, and unshielding to a regular EOA also reveals the recipient, currency, and amount, so amounts are transparent at the boundaries of the system. Internal user-to-user transfers fully hide amounts, giving confidentiality for balances held and moved within the shielded set.

Anonymity — Yes. For private transfers between users inside the privacy aggregators, both sender and recipient remain hidden — all qualities of the transaction are opaque. Shielding exposes the sender, currency, and amount but hides the recipient; unshielding to a regular EOA exposes the recipient, currency, and amount but hides the sender. Within the shielded set, both parties of a private transfer are anonymous.

Asset privacy — Yes. For private transfers between users inside the privacy aggregators, all qualities of the transaction — including the currency — remain hidden. Currency becomes visible at the boundaries: when shielding, the amount, currency, and sender are public, and at unshielding the currency and amount are public. The asset moved within the shielded set is not visible to external observers.

Plausible deniability — No. While the sender's outbound transfer goes to what looks like a fresh EOA, the Portal Broadcaster shortly afterwards deploys a Portal contract at that address via CREATE2 from the publicly known PortalFactory and forwards the funds into the Curvy privacy aggregator. Once the Portal is deployed and the funds are auto-shielded, the on-chain trail leads into known Curvy contracts, so the deposit is identifiable as a Curvy transaction. The fresh-address appearance is only transient, before the broadcaster runs.

Time-to-finality — N/A. Curvy is currently available on Arbitrum, Base, Ethereum, Optimism, Polygon, Binance Smart Chain, Linea, and Gnosis, with the privacy aggregator deployed on Arbitrum. Because Curvy is deployed as a set of smart contracts on these existing blockchains rather than operating as an independent chain, it does not define its own finality time. Transactions inherit the finality characteristics of whichever underlying chain they settle on—Arbitrum for aggregated private transactions, and each respective chain for portal operations.

Number of secrets — 2. The user holds an Ethereum wallet key to sign on-chain transactions, plus one Curvy-side root from which the viewing, spending, and Baby Jubjub private keys are deterministically derived. That root is either a FIDO2 passkey with PRF extension, or the signature of a fixed message under the wallet combined with a user password.

Deposit time — 0. Curvy generates a Portal address to which funds can be sent, and after the Portal address receives supported funds, the Portal Broadcaster deploys the Portal contract through the Portal Factory. From an on-chain observer's perspective, funds are sent to a regular EOA address to which a smart contract (Portal) is later deployed by the Portal Broadcaster, which then moves the funds to the Privacy Aggregator. The protocol does not impose any waiting period before a user can send funds to the pre-computed Portal address; the deployment happens reactively after funds arrive.

Withdraw time — 0. Proofs are generated by a Curvy-operated ZK Prover, and these are submitted on-chain to the aggregator. After verifying the withdrawal proof, the aggregator automatically initiates the unshielding to the recipient address in the same transaction. From an on-chain observer's perspective, funds simply move from the aggregator to the destination address. There is no delay or lock period imposed by the protocol.

Censorship resistance — No. Shielding requires going through a portal registered with the portal factory, and portal deployment is gated by the Curvy-operated Portal Broadcaster, which uses compliance scores from trusted analytics vendors to decide whether to deploy a portal for a given address. A flagged sender can therefore be blocked at entry, and there is no documented path for the user to bypass the Broadcaster and shield directly.

External network dependence — Yes, permissionless. Arbitrum is the only network that hosts the Privacy Aggregator smart contract, and on networks other than Arbitrum, the PortalFactory is configured to bridge funds through LiFi to Arbitrum so that they can be shielded. Portal Broadcasters are off-chain workers that scan announcements to identify Portal addresses that have received funds, in order to deploy a Portal. The protocol therefore relies on both the LiFi bridge network (a permissionless external bridge aggregator) and the Portal Broadcaster infrastructure to enable cross-chain shielding.

Escape hatch — No. Once funds are shielded into the privacy aggregator, the only path back out is an unshielding proof produced by the Curvy-operated ZK Prover and accepted by the aggregator's verifier contracts, both of which the operator controls. There is no mechanism that lets a user unilaterally exit shielded balances if the prover or operator becomes unavailable.

Upgradeability — Single admin. The aggregator is a UUPS proxy whose upgrade path is gated by a single owner address. The same owner can swap the insertion, aggregation, and withdrawal zk-SNARK verifiers, change the vault and portal factory, and reset the note and nullifier Merkle roots in emergency cases. There is no multi-sig, timelock, or DAO layer between the owner key and these operations.

Client-side proving — No. Aggregation and unshielding proofs are generated by a Curvy-operated ZK Prover service rather than on the user's device. The Curvy SDK in the user's browser prepares notes and ownership proofs and orchestrates the flow, but the SNARK proof itself is produced by the external prover and submitted on-chain. A decentralized proving design has been discussed but is not part of the live system.

Third-party inspectability — Yes. Producing aggregation and unshielding proofs requires the prover to know the private inputs of the transaction — the notes being spent, the amounts, and the destination. Today this prover role is operated by Curvy, so the operator is in a position to observe these plaintext inputs for every transaction it proves. This visibility is structural rather than something the user opts into by sharing a viewing key.

Implementation maturity — 3 : Mainnet for less than 1 year. Curvy went live on Arbitrum mainnet in July 2025, so as of April 2026 it has been deployed on mainnet for approximately 9 months.

Post-quantum secure — No. The actual implementation relies on secp256k1 for wallet-derived spending and viewing keypairs, Baby Jubjub for the in-circuit addressing key, and Baby Jubjub curves embedded in BN254-friendly zk-SNARK circuits — none of which resist Shor's algorithm. Curvy's docs reference research on post-quantum stealth address schemes, indicating awareness of quantum threats, but the live protocol does not implement post-quantum primitives.

Layer of enforcement — App. Curvy's pre-emptive compliance checks are enforced at the application layer: a Curvy-operated Portal Broadcaster uses security and compliance scores from analytics vendors to determine whether a Portal can be deployed, blocking shielding for flagged addresses. Retroactive compliance checks (currently work-in-progress) would also operate at the app layer, allowing a trusted entity to taint notes and restrict unshielding paths. KYC-gated registration is available only in custom enterprise deployments, not in the canonical Curvy Web App or protocol.

Enforcement entities — Admin. Pre-emptive compliance checks are performed by the Curvy-operated Portal Broadcaster, which uses scores from trusted analytics vendors (currently Global Ledger) to determine whether a portal can be deployed. Retroactive compliance checks grant a trusted entity, such as a DAO or an analytics vendor, the ability to taint malicious users even after funds have been shielded. The primary operational enforcement is carried out by Curvy (the admin/core team operating the Portal Broadcaster), while third-party analytics vendors provide the underlying compliance intelligence and DAOs are mentioned as potential trusted entities for retroactive actions.

Type of compliance — Programmatic policies, Selective disclosure. Curvy implements programmable per-deposit policies through trusted analytics vendors that provide security and compliance scores for addresses; the Curvy-operated Portal Broadcaster uses these scores to determine whether a portal can be deployed and assets shielded, aiming to prevent known malicious addresses from shielding funds. Retroactive compliance checks, currently in development, would enable a trusted entity to taint malicious users even after funds are shielded

— not yet live and therefore excluded from the value. Custom enterprise deployments support KYC checks that must pass prior to Curvy ID registration, but the canonical protocol does not feature this and KYC/KYB is therefore also excluded. Curvy IDs expose a viewing public key, enabling user-initiated selective disclosure via the viewing private key.

Point of enforcement — Deposit, Withdrawal. Pre-emptive compliance checks are performed by the Portal Broadcaster, which uses a security and compliance score from analytics vendors to determine whether a Portal can be deployed and assets shielded, enforcing compliance at the deposit step. Retroactive compliance checks enable tainting of malicious users after they have deposited and shielded funds, restricting them to unshield only to the exact same address from which funds were shielded, which enforces compliance at the withdrawal step. Both mechanisms are part of Curvy's two-sided compliance model, though retroactive compliance checks are not yet publicly available.

Selective disclosure: viewing entity — User, Voluntary third-party disclosure. The viewing private key can be shared by the user with a third party to gain access to their private transaction history that would otherwise be completely hidden, envisioned for proving transactional activity without giving control over funds. The viewing public key is necessary to construct proper stealth addresses and notes for the user. The user holds the viewing private key and can query their own transaction history, and voluntary delegation to trusted parties such as auditors is supported through key sharing. No involuntary third-party access or protocol-level disclosure mechanisms are documented.

Selective disclosure: viewing control — Pre-defined. The viewing private key can be shared by the user with a third party to gain access to their private transaction history, and is envisioned to be shared when the user wants to prove their transactional activity but does not want to give control over their funds. Curvy users' private keys are generated by deriving them from a deterministic source, such as FIDO2 Passkeys with PRF extensions or signature points from a third-party crypto wallet. The viewing key is generated once during user creation and there is no mechanism described for revoking, rotating, or scoping viewing permissions after the key has been shared—the recipient gains permanent access to the full transaction history.

Cryptographic verifiability — Yes. The Privacy Aggregator verifies zk-SNARK proofs on-chain for every state transition, validating that changes to the Sparse Merkle Trees of notes and nullifiers follow defined rules, and updates the tree roots only after successful proof verification. ZK proofs provide provable on-chain data and state transition rule integrity without exposing the exact state transitions. All three action types—shielding, aggregation, and unshielding—are verified as valid state transitions by the Privacy Aggregator.

Open source — Yes. The Solidity contracts repo is licensed under Apache-2.0, the SDK under MIT, and the kohaku wallet extension under GPL-3.0. All three critical components — contracts, SDK, and wallet extension — are published under OSI-approved licenses.

Private State Scalability — Infinity grow. Notes and Nullifiers are each arranged into a Sparse Merkle Tree, and with each proof submitted to the Aggregator smart contract, new Notes and/or Nullifiers are emitted through EVM events, and the roots of the SMT trees are updated on-chain. New Notes are created during shielding and aggregation operations, while aggregation consumes input Notes and produces output Notes. The trees accumulate entries monotonically as users shield funds and perform transfers, with no mechanism described for pruning or removing historical nullifiers or notes from the on-chain state. The Notes and Nullifiers data is essential so that every client can verify the validity of the SMTs and subsequently prove ownership of the Notes they wish to spend.

Client-side indexing — Always scanning. The Curvy App, using the open-source Curvy SDK, starts syncing notes from the public Notes Registry immediately upon starting, and the SDK simultaneously scans the synced notes with the user's private keys to detect which notes are secretly in their ownership and to decrypt their balances. Note ownership is completely hidden from on-chain observers, which means continuous scanning with private keys is required to identify which notes belong to the user. The SDK must attempt decryption on all notes because there is no public index by recipient address.

Private state model — UTXO-based state model. Curvy follows a UTXO-based model where each note represents an unspent output that is consumed by publishing its nullifier and producing fresh output notes. The privacy aggregator verifies proofs that validate state transitions of an ordered list of tuples called "Notes" and "Nullifiers," which are each arranged into Sparse Merkle Trees. During shielding, a new Note is created indicating a not-yet-spent balance of a certain currency. Aggregation combines one or many input Notes into one or many output notes, where multiple inputs can spawn multiple outputs, the sum of inputs and outputs must be equal, and the actor must prove ownership of all inputs.

Private Data Storage — Smart contracts. The privacy aggregator stores the Sparse Merkle Tree roots for notes and nullifiers in contract state, and emits each new note and nullifier as an EVM event so the data is persisted on-chain in the transaction logs. The Note Registry is an off-chain index of those events for ease of querying, but it is not the authoritative source — clients can rebuild the SMTs from chain history alone.

Access to DeFi — Access to internal DeFi ecosystem. Curvy supports private send, receive, and a private swap on shielded balances across ETH, WETH, USDT, UNI, WBTC, and USDC. The private swap operates directly on shielded notes inside the privacy aggregator. Arbitrary external DeFi protocols cannot be called while funds are shielded — users must unshield to interact with them — so composability is bounded to the in-protocol swap.

Programmability / Generality — Transfers and DeFi operations. Curvy is a privacy-first application that provides wallet features for receiving and sending assets privately. The platform currently supports a fixed set of tokens (ETH, WETH, USDT, UNI, WBTC, USDC) on multiple networks (Arbitrum, Base, Ethereum, Optimism, Polygon, Binance Smart Chain, Linea, Gnosis). The protocol surface consists of send, receive, and swap operations with no general-purpose

shielded contract execution layer — limited to private payments plus a swap feature, without support for arbitrary private smart contract logic or composable DeFi operations beyond the built-in swap.

Fluidkey

Fluidkey is a stealth address system on Ethereum that automatically generates a new Safe smart account for every incoming payment. It derives stealth addresses from user viewing keys following the ERC-5564 standard, surfacing them through a static ENS name (username.fkey.id) that resolves to a fresh stealth address on every query, ensuring funds arrive at an address that no observer can link back to the recipient.

Categories: Stealth addresses

Documentation: https://docs.fluidkey.com/

Confidentiality — No. Fluidkey does not provide confidentiality. Individual transaction amounts are publicly visible on-chain as standard ETH or ERC-20 transfers. While no external observer can aggregate a user's total balance (because funds are spread across unlinkable stealth addresses), each individual stealth address balance and transaction amount is fully visible.

Anonymity — Unlinkability. Stealth addresses provide unlinkability for the receiver: an outside observer sees funds sent to a fresh new address and cannot determine who controls it. However, the transaction shows the sender address and therefore the sender's balance and transaction history.

Asset privacy — No. The token type (ETH, USDC, etc.) is always visible on-chain in a Fluidkey transaction (normal on-chain transaction). Stealth addresses only obscure the link to the recipient's identity, not the asset being transferred.

Plausible deniability — Yes. Because every incoming payment lands on a fresh stealth address, an external observer cannot prove that a specific stealth address belongs to a particular receiver without access to the viewing key. From an outside perspective, Fluidkey transactions look like normal transactions going to addresses that have not been used before.

Time-to-finality — N/A. Fluidkey is deployed on Ethereum mainnet, Base, Optimism, Arbitrum, Polygon, and Gnosis. Time to finality is inherited from the underlying blockchain.

Number of secrets — 1. Users sign a single message with their Ethereum wallet, from which spending and viewing keys are derived using BIP-32 HD key derivation following the ERC-5564 standard. Therefore, 2 secrets which can be deterministically derived are required to use the Fluidkey protocol. The spending key controls fund transfers while the viewing key enables transaction scanning. No additional seed or mnemonic is needed beyond the Ethereum wallet.

Deposit time — 0. No mandatory deposit waiting period enforced by the protocol. From the outside, the deposit transaction is a normal Ethereum transaction sending funds to a fresh contract address

Withdraw time — 0. No mandatory withdrawal waiting period enforced by the protocol. Sending funds from a stealth account is a normal Ethereum transaction. If the stealth Safe account has not been deployed yet, deployment can be bundled in the same transaction, so there is no additional waiting time.

Censorship resistance — Yes. Stealth accounts are standard Safe smart accounts on public Ethereum networks. Any sender can transfer assets to a stealth address without interacting with Fluidkey's infrastructure. The open-source Fluidkey Stealth Account Kit allows anyone to independently derive stealth addresses and recover funds using only their signing key, without relying on Fluidkey servers.

External network dependence — No. Fluidkey is fully self-custodial, meaning only the user can access funds with their private keys, even without the Fluidkey interface. Multiple open-source recovery interfaces and tools are available to spin up independent web applications to interact with funds in stealth addresses.

Escape hatch — Instantly. Each stealth account is controlled by a stealth EOA. The stealth EOA is derived pseudo-randomly using the viewing key node shared by the user. This ensures that the user can independently replay all stealth addresses generated and recover funds without relying on Fluidkey.

Upgradeability — Immutable. Fluidkey stealth accounts are Safe smart accounts, a battle-tested and immutable smart account implementation. There is no admin key or upgrade mechanism that could alter the behaviour of deployed stealth accounts.

Client-side proving — N/A. Fluidkey does not use zero-knowledge proofs. The stealth address derivation relies entirely on elliptic curve Diffie-Hellman (ECDH) and BIP-32 key derivation, both of which are lightweight cryptographic operations feasible on any device including mobile.

Third-party inspectability — Yes. Fluidkey has read access to all incoming transactions and stealth account balances through the shared viewing key node. This access is required for Fluidkey to scan the blockchain and display the user's dashboard. Fluidkey cannot move funds (no spending key access), but can see all transaction history by default without explicit user consent per transaction. It is theoretically possible for a user to run their own indexer to scan the blockchain and infer their balance.

Implementation maturity — 4 : Mainnet for more than 1 year. Fluidkey launched its stable version in June 2024. Deployed on Ethereum mainnet, Base, Optimism, Arbitrum, Polygon, and Gnosis.

Post-quantum secure — No. Fluidkey's stealth address derivation is based on ECDH on the secp256k1 curve, which is vulnerable to Shor's algorithm and is therefore not post-quantum secure.

Layer of enforcement — None. Fluidkey does not enforce any AML/KYC or compliance rules at the protocol level. The stealth address system is permissionless: anyone can send to a stealth address without going through compliance checks.

Enforcement entities — None. There are no on-chain or protocol-level compliance entities in Fluidkey. Users can explicitly export their transactions for tax and accounting purposes.

Type of compliance — Selective disclosure. Fluidkey supports selective disclosure through its transaction export feature, which allows users to voluntarily reveal their full transaction history to third parties such as tax authorities or auditors.

Point of enforcement — None. There are no mandatory compliance enforcement points in Fluidkey. Any sender can send funds freely to a Fluidkey stealth account address and any receiver can use those funds with the Fluidkey interface or by running their own setup using community tools.

Selective disclosure: viewing entity — User, Voluntary third-party disclosure. Only the user can disclose their transaction history by sharing their viewing key. Although Fluidkey has read access to incoming balances via the shared viewing key node, all voluntary disclosures to third parties (excluding Fluidkey) are initiated and controlled by the user.

Selective disclosure: viewing control — Pre-defined. Fluidkey's viewing key grants full access to all transactions for a given account — it cannot be scoped or time-limited. The key structure is fixed at account setup time. Users share the viewing key as-is with no ability to restrict what the recipient can see.

Cryptographic verifiability — Yes. Fluidkey's stealth address scheme follows ERC-5564, using ECDH on secp256k1 for shared secret derivation and BIP-32 for hierarchical key derivation. Each stealth address is deterministically derived from the sender's ephemeral key and the recipient's viewing public key, making the derivation cryptographically verifiable by anyone holding the viewing key.

Open source — Yes. The Fluidkey Stealth Account Kit (core cryptographic functions) is open source under the MIT license. The Earn Module (DeFi yield integration) uses the GPL-3.0 license. Both are publicly available on GitHub.

Private State Scalability — Stateless. Fluidkey stealth accounts are counterfactually instantiated and each payment creates an independent smart account. There is no global shared state accumulation (no Merkle tree or registry that grows with usage). There could be a potential issue on deploying independent smart accounts every time a transaction is performed.

Client-side indexing — Always scanning. Continuous blockchain scanning is required because funds arrive at unlinkable stealth addresses. Fluidkey performs this scanning server-side using the shared viewing key node and notifies users when funds arrive.

Private state model — Account-based state model. Each incoming payment creates a new stealth Safe smart account at a deterministic address. Funds are distributed across independent accounts, each controlled by a derived stealth EOA. This is an account-based model where each stealth account holds mutable state.

Private Data Storage — Smart contracts. Each stealth account is a Safe smart account deployed on-chain that holds user's funds. Contracts are deployed lazily (counterfactual instantiation), it is only deployed on-chain when the user wishes to move funds out.

Access to DeFi — Unlimited access to DeFi applications. Stealth accounts are standard Safe smart accounts that can interact with any Ethereum DeFi protocol. The Fluidkey interface surfaces a curated Earn feature for yield-generating protocols, but users are not restricted to it — any DeFi interaction possible with a Safe account is available.

Programmability / Generality — Only payments. Fluidkey is designed primarily for private payments. The Fluidkey interface and protocol are focused exclusively on payment use cases, even though the underlying Safe smart accounts are theoretically programmable.

Hinkal

A privacy protocol using stealth addresses and zk-SNARKs to enable confidential token transfers and shielded DeFi interactions on EVM chains, with built-in KYT compliance enforced at deposit and withdrawal.

Categories: Shielded pool, Stealth addresses, Zero Knowledge Proofs (ZKPs)

Documentation: https://hinkal-team.gitbook.io/hinkal

Confidentiality — Yes. Hinkal hides both balances and transfer amounts. Assets are deposited into a shared shielded pool and tracked as encrypted UTXOs, so external observers cannot determine how much a user holds or transfers. Each transaction consumes a note by adding its nullifier to the state and creates new notes by adding new values to the commitment tree. To ensure a recipient receives the secret information required to spend their commitments, the sender encrypts the secrets of the commitment sent to the recipient.

Anonymity — Yes. Hinkal uses stealth addresses derived per user using their canonical public key so that on-chain interactions cannot be linked to a user's identity. The shielded pool hides the sender and receiver of every internal transfer due to its note-based UTXOs model. Every transaction uses input notes (adds its nullifiers to the tree) and creates new notes (add commitments to the commitment tree) pointing to new fresh stealth addresses. A commitment is defined as the Poseidon hash of the token address, amount, stealth address (recipient) and timestamp.

Asset privacy — Yes. There are three transactions types in Hinkal: shield (deposit), unshield (withdraw) and private transfer. As expected when a user shields and/or unshields tokens, the asset type (e.g. native token, ERC20, etc) is revealed because the Hinkal contract or the recipient end up with the tokens in the public on-chain registry. In a private transfer, the sender nullifies some commitments and creates new commitments. There is no information about the asset type published on-chain. Therefore asset privacy is preserved.

Plausible deniability — No. Deposits and withdrawals are direct interactions with the Hinkal smart contracts, which are publicly visible on-chain. It is therefore apparent that a user is engaging with the Hinkal privacy protocol. On private transfers, senders can use relayers to hide their public addresses but in order to do so they would initially have deposited tokens into the Hinkal contract.

Time-to-finality — N/A. Hinkal is deployed on Solana, Tron, Ethereum, Polygon, Arbitrum, Optimism, Base, and Arc. Time to finality is inherited from the underlying blockchain.

Number of secrets — 1. Hinkal requires a spending key and a viewing key to operate, but both derive from a single Ethereum wallet signature. The spending key is produced deterministically from an ECDSA signature (secp256k1), and the viewing key (Curve25519) is derived from the spending key using libsodium. Stealth addresses use the BabyJubjub curve with Poseidon hashing for commitments. Only the Ethereum wallet key needs to be stored independently.

Deposit time — 0. Users that deposit into Hinkal will instantly be able to interact: private transfer, privately interact with other smart contracts, withdraw. But it is recommended to define a time window during which the transaction executes to reduce linkage between intent and settlement

Withdraw time — 0. Hinkal does not impose a mandatory withdrawal waiting period. Users can withdraw shielded assets to a public address instantly, either self-initiated or via a relayer. It is recommended to withdraw after a time period in order to avoid meta data and pattern linkage

Censorship resistance — No. Hinkal enforces KYT compliance at the smart contract layer through Chainalysis compliance components. Deposits are screened in real-time and blocked if transactions are involved in sanctioned or blacklisted addresses. This means that participation into Hinkal protocol and features is restricted to only users that have not been sanctioned or are not blacklisted according to the compliance enforcement organization (Chainalysis).

External network dependence — No. Hinkal operates entirely on EVM-compatible chains without relying on external networks for consensus or transaction validation. Users generate ZK proofs off-chain and submit them directly to the smart contracts. An optional relayer service exists for gas abstraction but is not required.

Escape hatch — No. If an address is sanctioned or blacklisted by Chainalysis, withdrawal requests from that address will be blocked at the smart contract level. Users would need to work with regulators and the Hinkal team to resolve access. This is a design trade-off for institutional compliance.

Upgradeability — Multi-sig. Hinkal does not have a governance token at the moment but there are plans to launch a token in the near future. At the moment any upgrades to the protocol would be done by Hinkal internal team which likely uses a multi-signature wallet to introduce new changes on-chain. TODO: is this true, it looks like the answer is uncertain and source doesn't prove it either way

Client-side proving — Yes. ZK proofs for deposits and withdrawals are generated on the user's device. Simple private transfer transaction ZK proofs can also be generated locally but more complex ones (involving smart contract interactions, multiple tokens, etc) could take longer. Therefore Hinkal implements an external proof generation mechanism using AWS Nitro enclave. There is an option to turn on "Extra Security" and generate the ZK proofs inside the user's browser at a slower speed. TODO: Source doesn't mention AWS. Need to verify

Third-party inspectability — No. If the "Extra Security" option is turned off, all private inputs for the ZK proofs (token address, amount, stealth address, etc.) are sent in plaintext to the AWS Nitro enclave. We can assume that the enclave (TEE) will not leak data to any third party but it is important to mention that there are known vulnerabilities around TEE that might allow an attacker to see the inside data. An alternative way to solve this issue is to turn on "Extra Security" and generate the proof locally. TODO: Source doesn't mention AWS. Need to verify

Implementation maturity — 3 : Mainnet for less than 1 year. The Hinkal shielded-pool contract was first deployed on Ethereum mainnet in June 27, 2025. The protocol has been live on mainnet for less than one year at time of evaluation.

Post-quantum secure — No. Hinkal's privacy guarantees rely on Elliptic-Curve Cryptography (ECC) and zk-SNARKs (Groth16 / similar pairing-based schemes), which are not secure against a sufficiently powerful quantum computer due to Shor's algorithm. It is worth pointing out that the harvest-now-decrypt-later problem is present in Hinkal due to the transaction data encryption using a Curve25519-based keypair.

Layer of enforcement — Protocol/chain. Compliance is enforced at the protocol layer through Chainalysis compliance components integrated into the smart contracts. The Hinkal front-end and relayer infrastructure also check addresses against KYT providers before allowing deposits or withdrawals. SDK integrations can rely on integrator-verified compliance, traceability configuration, and zkTLS/KYC via external partners.

Enforcement entities — Third party. KYT checks are delegated to a third-party compliance provider (Chainalysis) integrated into the Hinkal smart contracts directly. Chainalysis compliance components are on-chain smart contracts that authorize or not a specific address or transaction depending on external automated analysis.

Type of compliance — Programmatic policies, Selective disclosure. Hinkal enforces programmable per-transaction policies via Chainalysis compliance components integrated into the smart contracts; the chain-analytics signal is a data input to the policy layer (not a separate compliance category). Screening applies at deposit and withdrawal, where public addresses are visible. Private transfers within the shielded pool are note-to-note and do not expose public addresses to the compliance layer. Users can also voluntarily share their viewing key with third parties for selective disclosure.

Point of enforcement — Deposit, Withdrawal. Compliance checks are applied when assets enter the shielded pool (deposit) and when they leave (withdrawal), blocking flagged addresses at either boundary. Private transfers within the pool are note-to-note operations between stealth addresses and do not pass through the Chainalysis compliance contract directly.

Selective disclosure: viewing entity — User, Voluntary third-party disclosure. Users can view their own transaction history via their viewing key (self-view). They can also voluntarily share this viewing key with third parties such as regulators, auditors, or wallet managers, granting read-only access to their shielded transaction history.

Selective disclosure: viewing control — Pre-defined. Viewing access follows a pre-defined key-derivation scheme. There is no programmable, scope-limited viewing key; disclosure covers the full history accessible by the derived key. There is no update function to change the viewing key because it is derived from the spending key which is scoped to one per user.

Cryptographic verifiability — Yes. Correctness of all shielded operations is guaranteed by zk-SNARK proofs (Groth16 or similar pairing-based scheme on BN254) verified on-chain. Commitments use Poseidon hashing over the BabyJubjub curve, and viewing keys use Curve25519 encryption.

Open source — No. Hinkal's GitHub organization contains demo apps and forks but the main smart contract repository is not publicly accessible (404 as of April 2026). No open source license information was found for the core protocol contracts.

Private State Scalability — Infinity grow. Hinkal uses an append-only Merkle tree to store UTXO commitments and nullifiers. Every shielded operation adds new leaves, so state grows unboundedly over time. A removal-supporting Merkle tree variant is supported for certain interactions.

Client-side indexing — Partial scanning. Users must scan chain events to discover incoming stealth transfers and reconstruct their shielded balance. Hinkal provides tooling (Hinkal Receive) for SDK integrators to build this scanning component. The wallet performs scanning internally for events related to the user's stealth addresses.

Private state model — UTXO-based state model. Hinkal tracks private balances as encrypted UTXOs (commitments and nullifiers) stored in Merkle trees.

Private Data Storage — Smart contracts. All UTXO commitments, nullifiers, and Merkle tree roots are stored directly in the Hinkal smart contracts deployed on supported EVM chains. The commitment creation or nullifier addition (nullified) actions are notified using events. This requires the wallet provider or integrator to have a RPC event scanning mechanism to keep the user shielded balance up to date in real-time.

Access to DeFi — Access to external, but limited choice of DeFi protocols. Hinkal supports shielded swaps and interactions with a curated set of DeFi protocols directly from the shielded pool, but the choice is limited to integrations explicitly added by the Hinkal team.

Programmability / Generality — Partial programmability. Hinkal supports private token transfers and integrates with a curated set of DeFi protocols through hook contracts that execute before or after token exchanges. There is no support for arbitrary private smart contract logic, but the hook system enables structured DeFi interactions (swaps, lending) from within the shielded pool.

Houdini Swap

Houdini Swap is a non-custodial cross-chain liquidity aggregator that obtains privacy by routing each swap through two independent off-chain CEX partners and three separate blockchains, so no single counterparty sees the full path from source to destination. It is not a mixer: liquidity is not pooled and no zero-knowledge proofs are used; privacy comes from partitioning knowledge across routing hops. The service aggregates DEXes and bridges across 100+ chains and curates CEX partners that run industry-standard AML programmes.

Categories: Cross-chain swap aggregator

Documentation: https://docs.houdiniswap.com/

Confidentiality — No. Balances and transfer amounts are transparent. Each swap consists of multiple hops routed through several intermediaries: For example, token A is sent from the source wallet and routed to a non-custodial exchange, where it is swapped into a temporary, privacy-centric Layer 1 intermediary token. This intermediary token is then routed to a second non-custodial exchange and delivered as token B to the recipient, meaning every hop settles as an ordinary on-chain transfer with visible amounts. There are no shielded commitments, notes, or encrypted amounts at both ends of the swap. The privacy focussed L1 token acts as the buffer that breaks the link in the chain — privacy comes from breaking the sender-receiver link across hops, not from hiding values.

Anonymity — Unlinkability. Sender and receiver addresses are unlinkable on-chain. Each swap is routed through separate non-custodial exchanges with a temporary layer-1 token as an intermediary, and the sender and receiver never appear in the same transaction trail. Each leg of the transaction appears to terminate independently, so to an outside observer each leg looks like a standard, unrelated transaction. Addresses themselves remain visible on their respective chains.

Asset privacy — No. The asset being transferred is visible to observers at every hop. Each leg routes funds through a non-custodial exchange, with token A swapped into a temporary layer-1 intermediary on the source side, and the intermediary then routed to a second exchange and swapped into the desired token B before delivery. Each trade starts and settles with a specific public asset.

Plausible deniability — Yes. The entry leg of a Houdini Swap is a plain transfer from the user's wallet to a partner CEX deposit address — no Houdini-owned contract is called and no privacy-specific calldata is involved. Routing is automated via exchange protocols rather than a Houdini Swap contract. Per the docs there are no deterministic on-chain markers showing a transaction is

a Houdini Swap, and to an outside observer each leg looks like a standard, unrelated transaction. Caveat: a determined observer who enumerates Houdini's CEX partner addresses over time could infer routing from patterns, but that is heuristic inference rather than a protocol-level marker.

Time-to-finality — N/A. Houdini Swap is a routing service rather than its own chain or rollup, so finality is inherited from whichever source and destination networks a given swap touches. Houdini does not execute swaps itself. Each route is handled by the underlying protocol or exchange partner selected for that transaction. Bridging or swaps are offered between most EVM chains, Bitcoin, Solana and SUI. Effective finality therefore varies per swap and equals the slower of the two endpoint chains' finality plus off-chain CEX-hop latency.

Number of secrets — 1. The user must store one secret to use Houdini: the pre-existing wallet key that signs the source-leg deposit transaction on the originating chain. Houdini is a non-custodial, cross-chain swap aggregator that does not hold user funds, maintain balances, or act as a counterparty, so it issues no viewing key, spending key, nullifier, or shielded account of its own. Routing forwards token A from the source wallet to a non-custodial exchange and on to a second exchange for delivery — none of these legs require any user-held secret beyond the source wallet's signing key.

Deposit time — 0. No protocol-enforced waiting period applies before a user can deposit. The user sends funds to a Houdini-provided deposit address as the first step of routing, with no pre-deposit delay in the routing sequence.

Withdraw time — 0. No protocol-imposed withdrawal delay exists. The destination token is delivered directly to the recipient address as the final step of routing. After multi-hop CEX routing, the final token arrives at the user's destination address, with no separate withdrawal action required by the user. The end-to-end timing windows — 15–45 minutes for private swaps and 3–30 minutes for standard swaps — reflect CEX processing and routing latency rather than a payout lock between leg completion and delivery.

Censorship resistance — No. Swaps can be blocked by multiple entities. Partner CEXes must use tools like Chainalysis to screen incoming deposits for links to sanctioned wallets or criminal activity, and they prevent transactions from OFAC-listed entities and jurisdictions, so a CEX hop can reject a deposit or payout mid-route. Houdini itself imposes structural gates: swaps over $100,000 are not facilitated, access from Tor and restricted jurisdictions is blocked, and users connecting from sanctioned countries are geo-blocked. Because routing depends on these vetted intermediaries with no permissionless fallback, a refusal at either layer halts the transfer.

External network dependence — Yes, permissioned. Houdini's Private Swap routing depends on external networks it does not operate: partner CEXes for the multi-hop privacy path, and on-chain DEX aggregators, cross-chain bridges, and intent-based protocols for the on-chain path. The CEX partner set is curated rather than open — partners must pass Houdini's Know Your Business

due diligence before integration — so the dependency is on a permissioned set of providers, not an open network. The Houdini routing engine itself only analyses routes and orchestrates the order lifecycle; all settlement security is inherited from these external venues.

Escape hatch — N/A. The escape hatch property does not apply here because there is no protocol-controlled pool or shielded balance to exit from. The routing is non-custodial and executed through exchange protocols rather than a Houdini Swap contract, and each swap passes through two separate non-custodial exchanges with a temporary intermediary asset acting as a buffer between legs. If a partner exchange fails to deliver during the brief window between legs, recovery is a customer-service matter with that venue, not an on-chain withdrawal path. According to the docs, Houdini Swap offers a 24/7 human support portal on Telegram.

Upgradeability — Single admin. Upgrades are controlled unilaterally by Houdini as the operator of an off-chain routing service. The routing engine analyses available routes, calculates optimal paths, and manages order lifecycle — all of which run off-chain and can be modified by the operator at any time without user consent or on-chain governance. The set of integrated CEX partners and on-chain venues is curated by Houdini, so partner list, pricing, and routing strategy are changeable server-side. There are no Houdini-deployed smart contracts with a formal upgrade path, so upgradeability is based on operator control over the SaaS backend rather than a proxy or governance contract.

Client-side proving — N/A. Houdini Swap has no proving component. Privacy is produced by routing funds through two separate non-custodial exchanges with a temporary Layer 1 token as an intermediary, not by any zero-knowledge proof system. The process is automated via exchange protocols off-chain, meaning there are no notes, nullifiers, or proofs generated on either the client or a server.

Third-party inspectability — Yes. Third parties can inspect user transaction information. Each non-custodial exchange partner observes the leg of the swap routed through it, including source address, amount, and payout address, and partners maintain records for their side of each transaction to support regulatory and law-enforcement investigations. Under valid legal process these partners can reconcile their respective logs to reconstruct the full source-to-destination path, and Houdini adheres to a legally compliant process for data requests. The routing engine additionally has visibility into orchestration metadata, with transaction data retained for 72 hours before auto-deletion.

Implementation maturity — 5 : Mainnet for more than 2 years. Houdini Swap is a non-custodial, cross-chain swap aggregator that routes token swaps across decentralized exchanges, cross-chain solvers, and non-custodial exchange partners. Maturity reflects continuous service uptime and integration breadth rather than a deployed smart-contract codebase, since there are no proprietary liquidity pools, order books, or exchange contracts. The service operates at scale across many chains and partners. The first-launch date has not been verified against block-explorer first-transaction evidence or a dated announcement.

Post-quantum secure — No. The protocol adds no cryptography of its own, instead relying on each leg's underlying chain signatures (secp256k1 ECDSA on EVM chains, Ed25519 on Solana) plus TLS to partner exchange APIs, none of which resist quantum attacks.

Layer of enforcement — App. Compliance enforcement sits at the application layer, operated by Houdini's curated set of centralised exchange partners rather than by any on-chain contract or asset-level control. AML and anti-terrorist-financing liability for users is delegated to the exchange partners, who employ real-time, risk-based transaction-monitoring systems to detect and investigate suspicious activity.

Enforcement entities — Third party. Compliance enforcement sits primarily with third-party exchange partners, with Houdini's internal compliance team acting as orchestrator and auditor. AML/ATF liability for users is the responsibility of the exchange partners, who run real-time risk-based screening to detect suspicious activity. These partners screen against sanctions lists and refuse transactions tied to listed entities or jurisdictions, and maintain records for their side of each transaction to support regulatory investigations. Houdini retains a Compliance Officer plus external advisors, but the actual screening, blocking, and law-enforcement response is executed by the exchange partners.

Type of compliance — Programmatic policies. Compliance operates as programmatic per-transaction policies enforced at the partner-exchange layer, with real-time AML/AFT screening using tools like Chainalysis to check incoming deposits against sanctioned wallets and prevent transactions from OFAC-listed entities and jurisdictions. Partner onboarding uses a KYB-style risk assessment — partners must pass "Know Your Business" due diligence proving active AML/OFAC compliance.

Point of enforcement — Deposit, Withdrawal. Enforcement occurs at both the deposit and withdrawal legs, handled by the non-custodial exchange partners that bracket the swap. Partner exchanges employ real-time, risk-based transaction-monitoring systems to detect suspicious activity, and block OFAC-sanctioned senders, which applies symmetrically to refusing payout destinations on the exit leg.

Selective disclosure: viewing entity — Involuntary third-party disclosure. Selective disclosure occurs via law-enforcement requests directed at exchange partners, with no user-facing viewing mechanism. When approached by law enforcement, Houdini adheres to a legally compliant process, responding to valid requests while safeguarding user privacy. No cryptographic viewing key, self-view portal, or voluntary delegation mechanism exists — disclosure happens off-chain through legal process against the CEX legs.

Selective disclosure: viewing control — Pre-defined. Houdini Swap has no dedicated viewing-key or selective-disclosure mechanism. Off-chain disclosure happens through legally compliant responses to law-enforcement data requests and through CEX internal records. The viewing control is pre-defined by the CEX partners because they can view transaction details at any step of the process.

Cryptographic verifiability — No. No cryptographic proofs guarantee the correctness of a Private Swap's routing or settlement. Execution relies on two separate non-custodial exchanges, meaning settlement depends on off-chain CEX bookkeeping rather than on-chain verifiable computation.

Open source — No. The public GitHub organisation for Houdini Swap publishes only a client-facing API example set, not the core routing, matching, or compliance logic. The proprietary routing engine and CEX partner integrations that implement the privacy model are not published, and no OSI-approved licence covers the protocol internals.

Private State Scalability — Stateless. Houdini Swap is stateless with respect to protocol-specific private data, since it maintains no note tree, nullifier set, shielded pool, or encrypted state. Each swap route leaves no persistent private state.

Client-side indexing — No scanning. Houdini Swap requires no client-side chain scanning. Balances on the source and destination accounts are held in standard wallet addresses readable by any block explorer or wallet software. with no notes, commitments, or encrypted state to decrypt or track.

Private state model — N/A. Houdini Swap does not maintain any protocol-level private state. User balances remain ordinary wallet balances on whichever host chain they reside on. Most supported chains and the majority of swap volume sit on account-model networks, so so you could argue the closest match is the account-based model, though this property does not really apply to an off-chain routing service.

Private Data Storage — N/A. Houdini Swap does not maintain any private transaction data of its own — there are no on-chain privacy commitments, shielded notes, or off-chain encrypted state tied to a chain commitment. Each leg is an ordinary on-chain transfer; per-leg transaction records reside inside each partner exchange's internal systems for regulatory purposes, while the routing engine keeps order metadata off-chain to orchestrate legs. The property does not apply to an off-chain routing service.

Access to DeFi — No access to DeFi. Houdini operates as a swap router that delivers plain destination tokens to a user-controlled address, with no protocol-issued shielded asset, wrapper, or note format that could plug into DeFi. After a swap, any subsequent DeFi use is whatever the recipient does with the plaintext token on the destination chain.

Programmability / Generality — Only payments. Houdini Swap's programmability is limited to payments only, specifically token swaps and cross-chain transfers. The protocol exposes three swap primitives — private, no wallet connect, and DEX — all of which route token movements rather than execute arbitrary private logic. All other features are built on top of these swap primitives, so there is no private smart-contract layer, no general private computation, and no notes or state objects that user logic could operate on.

Nullmask

Nullmask is a privacy layer that sits between an ordinary wallet (e.g. MetaMask) and a chain via a custom RPC proxy. Users sign a single authorization message to derive viewing, nullifying and encryption keys. The proxy converts a user's normal transactions into shielded notes and produces UltraHonk zk-SNARK proofs for deposits, transfers and withdrawals. All shielded transactions processed by the RPC proxy are sent to the Nullmask contract to be executed and registered. Shielded state is represented as encrypted notes on-chain alongside a key registry. Users trial-decrypt notes to find their own. Retroactive compliance allows flagged deposits to be revoked.

Categories: VPN (Private Intents)

Documentation: https://docs.nullmask.io/

Confidentiality — Yes. Shielded balances exist as encrypted UTXO-style notes rather than public ERC-20 balances, and transaction token and amount stay private for shielded transfers, although token and amount are visible for shielded swaps and withdrawals. All incoming and outgoing transfers are completely private, and no external observer can decrypt the fields of shielded transactions. Both the per-user balance (as a set of encrypted notes) and in-protocol transfer amounts are hidden from external observers, with amounts only becoming visible at the public boundary of swaps and withdrawals.

Anonymity — Yes. Both sender and recipient remain hidden for shielded transfers within the protocol. All incoming and outgoing account transfers are private, and no external observer can decrypt the fields of shielded transactions. Deposit and withdrawal operations reveal public addresses, but transfers inside the shielded set conceal both parties.

Asset privacy — Yes. For shielded transfers, the asset identity is hidden inside the protocol. Each note carries a field identifying the token (the zero address denotes native ETH), and that token identifier is bound into the value commitment via a Poseidon hash, which is in turn hashed into the final note commitment stored on-chain. Since only the commitment is published, the specific asset identity is hidden alongside the amount within the shielded pool. Asset type is visible at deposit and withdrawal boundaries.

Plausible deniability — No. Nullmask does not offer plausible deniability for deposits: entering the pool requires interacting with a known privacy contract. Deposits are screened by a guard service that monitors pending deposits on the contract, approves legitimate ones by adding note commitments to the Merkle tree, and rejects flagged ones, meaning the deposit is a visible call to the Nullmask contract rather than a plain transfer. The on-chain contract verifies ZK proofs, manages the note commitment Merkle tree, and tracks spent nullifiers, making any interaction with it identifiable as privacy-pool activity on-chain.

Time-to-finality — N/A. Nullmask is an application-layer protocol with no independent consensus, so finality is inherited from whichever underlying chain a user transacts on. It is deployed on Ethereum Mainnet, MegaETH, Arbitrum One, Base, and BSC, with Ethereum Sepolia as the testnet.

Number of secrets — 1. Only the wallet key itself needs independent storage. All cryptographic keys are derived deterministically from a single wallet signature, and because modern wallets use deterministic signatures (RFC 6979), the keys can be recovered by re-signing a fixed message. From that signature a seed is produced via Poseidon hashing, and the nullifying key, incoming viewing key, and outgoing viewing key are each derived as further Poseidon hashes of the seed. No additional secret material beyond the wallet key needs independent storage.

Deposit time — 15-30 seconds. When the user makes a deposit, the funds are held by the contract and a pending event is emitted. A guard service then screens the depositor address and either accepts or rejects it. This screening gates acceptance into the Merkle tree. It usually takes 15-30 seconds but it depends on the guard service.

Withdraw time — 0. Withdrawals complete within a single transaction. The execution flow verifies the zero-knowledge proof, spends note nullifiers, records the transaction nullifier, adds change note commitments to the Merkle tree, pays the relayer fee, and transfers the withdrawal amount to the recipient — all within one call. There is no time-lock, challenge window, or queued withdrawal step in the settlement path.

Censorship resistance — No. Deposits require approval from a privileged guard address. The guard screens depositor addresses with chain-analysis tools and is the only party able to approve or reject a pending deposit, so there is no permissionless path into the pool for a flagged address. The contract is upgradeable: the guard address can be swapped by the current guard or by the upgrade controller admin, and the verifier contracts can be replaced with upgrade controller authorisation, so a single admin can rotate the guard or swap verifiers to withhold inclusion of valid deposits. Post-deposit shielded transfers and withdrawals carry no caller restriction, so the deposit gate is the binding constraint.

External network dependence — Yes, permissioned. Nullmask relies on several off-chain services operated by the project team. An RPC proxy sits between the wallet and the chain, handling key management, transaction shielding via ZK proof generation, balance tracking, and state isolation. A relayer receives shielded transaction data from the proxy, submits transactions to the contract, and pays gas, hiding the sender's IP and identity. A guard monitors pending deposits, screens depositing addresses using chain-analysis tools, and approves or rejects them. These are permissioned operator roles.

Escape hatch — Instantly. Withdrawals settle in the same transaction as proof verification, with no protocol-imposed delay. After the on-chain verifier validates the proof, the contract spends the nullifiers, records the transaction nullifier, inserts the change commitments, pays the relayer fee, and transfers the withdrawal amount to the recipient. Exit depends on the verifier contract, which sits behind an upgradeable proxy with upgrade authority delegated to a controller — a controller-driven upgrade could alter the verifier.

Upgradeability — Single admin. The main contract is a UUPS upgradeable contract, and the upgrade permission mechanism itself is upgradeable. The current upgrade controller is held by a single admin (the deployer), with that same authority gating verifier swaps and other privileged setters. Migration to a timelock, multisig, or DAO is described as a future possibility rather than a live arrangement. The on-chain admin address has not been independently verified to confirm whether the deployer key is an EOA or a multisig wrapper.

Client-side proving — No. By default, proofs are generated by the RPC proxy: it parses transaction intent, selects funding notes, and generates ZK proofs server-side, accessed via a web page where the server operator can observe user transfers. The protocol uses the UltraHonk proof system, which the docs note can prove an EIP-1559 transaction and verify its ECDSA signature on consumer-grade hardware in roughly two seconds, so client-side proving is supported. A self-hosted path running on mobile or as a browser extension is documented as an alternative privacy-preserving deployment, but it requires installation and is not the default flow for the typical user.

Third-party inspectability — Yes. The RPC proxy parses transaction intent, selects funding notes, and generates ZK proofs, meaning it processes recipient, amount, and token data in plaintext before shielding. Per-user note storage and key storage is held by the proxy. The relayer, by contrast, never sees transaction contents and only forwards the proof and public inputs.

Implementation maturity — 3 : Mainnet for less than 1 year. Nullmask is deployed on Ethereum Mainnet plus several other production chains (MegaETH, Arbitrum One, Base, BSC) alongside a Sepolia testnet, with a public web app. The mainnet deployment date The mainnet deployment date is February 3rd 2026.

Post-quantum secure — No. The protocol's entire cryptographic stack relies on classical primitives vulnerable to quantum attacks. It uses the UltraHonk proof system via the Barretenberg prover over the BN254 elliptic curve, the Grumpkin embedded curve whose base field equals the BN254 scalar field, the Poseidon2 hash function designed for zk-SNARK circuits, and secp256k1 ECDSA for Ethereum transaction signatures verified inside the circuit. Discrete-log-based curves (BN254, Grumpkin, secp256k1) are broken by Shor's algorithm, and SNARK-friendly hashes offer no post-quantum guarantees.

Layer of enforcement — Protocol/chain. Compliance is enforced at the protocol contract layer: every deposit to the Nullmask contract must be verified and approved by the guard service before the deposited funds become available as shielded notes, with the contract refusing to add a note commitment to the Merkle tree absent that approval. Only the guard service is authorised to mark deposits as approved or rejected. The gate is on-chain and the underlying chain-analysis screening runs off-chain.

Enforcement entities — Third party. Compliance enforcement is gated by chain-analysis tools whose verdict the Nullmask-operated Guard service acts on. The Guard monitors pending deposits, screens addresses with external chain-analysis tools, and approves or rejects them

based on that screening. the Guard is the operational executor.

Type of compliance — Programmatic policies, Proof of innocence (POI) / ASP. Nullmask enforces two mechanisms. First, a guard service performs analytics-based screening on every deposit — each must be verified before the deposited funds become available as shielded notes, with the guard evaluating links to illicit activity, sanctioned entities, and money-laundering signatures. Second, a retrospective exclusion set mechanism uses revocation keys: the guard publishes a revocation key with each approved deposit, and if a deposit is later flagged the secret key is released so any participant can prove whether their notes are tainted.

Point of enforcement — Deposit. Compliance is enforced at the deposit boundary. Every deposit must be verified and approved by the Guard service before the deposited funds become available as shielded notes, with rejection resulting in a refund. Once a note is in the pool, internal shielded transfers, withdrawals, and shielded swaps carry no compliance step. A retrospective revocation-key mechanism allows participants to derive a proof of disassociation if a deposit is later flagged, but this is a user-side opt-in rather than a protocol gate fired at any specific step.

Selective disclosure: viewing entity — User, Voluntary third-party disclosure. Users hold a viewing-key tuple — a public key, nullifying key, incoming viewing key, and outgoing viewing key — where the incoming viewing key decrypts incoming notes and the outgoing viewing key decrypts receipts of sent notes, giving self-view of balance and transaction history. Voluntary delegation is possible because the viewing key grants view access to the account's transaction history and can be shared with a third party such as an auditor, who would then be able to decrypt notes without any spending capability.

Selective disclosure: viewing control — Pre-defined. All cryptographic keys are derived deterministically from a single wallet signature, the keys can be recovered by re-signing a fixed message. The viewing key set is derived once via Poseidon hashing of the signature seed, with no mechanism for scoping, rotation, or revocation. It grants view access to the account's transaction history, so sharing it gives the recipient full read access.

Cryptographic verifiability — Yes. Transaction correctness is enforced cryptographically: every shielded action requires a valid ZK proof verified by an on-chain verifier contract, and the proof cannot be forged without breaking the UltraHonk proof system, with Ethereum signatures verified inside the ZK circuit via secp256k1 ECDSA. Proofs are verified on-chain. The verifier contracts are upgradeable.

Open source — No. The documentation lists deployed addresses and comprehensively describes the protocol architecture, smart contract interfaces, and RPC API in detail. The documentation states that contracts and circuits are open source, but no public source repository or open source licence is referenced.

Private State Scalability — Infinity grow. Private state grows without bound with usage. Every shielded action appends a new note commitment to an on-chain Merkle tree and publishes a nullifier that is permanently retained. The contract maintains a set of all spent nullifiers and rejects

any transaction that attempts to spend a note whose nullifier has already been recorded.

Client-side indexing — Always scanning. User devices (via the RPC proxy) must continuously scan the chain to identify incoming notes, since ownership is not indexed by recipient address. The RPC proxy handles balance tracking by scanning the blockchain for new notes, trial-decrypting them, and maintaining shielded balances, with a dedicated notes scanner service that monitors blockchain events for incoming notes.

Private state model — UTXO-based state model. Nullmask uses a UTXO-based private state model: the shielded account state is managed as a UTXO set, with each UTXO called a note — an encrypted UTXO holding value, token type, and owner information. Note commitments are published to an on-chain Merkle tree, and spending a note publishes its nullifier, with the contract maintaining a set of spent nullifiers to prevent double-spends.

Private Data Storage — Smart contracts. Authoritative private transaction data lives on-chain in the protocol's own contracts. Contract storage holds the note-commitment Merkle tree, the receiving-key registry tree, the spent note nullifier mapping, and the transaction nullifier mapping, while encrypted note ciphertexts are emitted in the NoteAdded event as a fixed-size encrypted payload, making the full encrypted payload part of chain history rather than any off-chain store. The RPC proxy maintains a per-user local cache of decrypted notes, keys, and a synced Merkle tree for performance and UX, but that is a derivative store of on-chain data, not the source of truth.

Access to DeFi — Access to external, but limited choice of DeFi protocols. Nullmask supports shielded swaps via Uniswap V2. The pool contract, which holds public token balances, executes the Uniswap V2 swap on the user's behalf and credits them with shielded output notes in the new token, so the user never has to manually unshield, swap, and reshield. The tokens are sent to Uniswap for the swap, but the users balance update happen as regular shielded note creation, rather than as a public balance change for the user. Other DeFi primitives (lending, staking, derivatives) are not supported.

Programmability / Generality — Transfers and DeFi operations. Nullmask supports payments and specific DeFi operations. A user can deposit, transfer tokens, swap tokens, and withdraw. There is no generalised private-state execution or arbitrary contract interaction.

Privacy Pools

A privacy protocol that extends Tornado Cash unlinkability concept with association set providers (ASPs), allowing users to prove membership in compliant subsets without revealing their identity.

Categories: Mixer, Zero Knowledge Proofs (ZKPs)

Documentation: https://docs.privacypools.com/

Confidentiality — No. Privacy Pools uses the same the note-based architecture as Tornado Cash. Deposit and withdrawal amounts are publicly visible on-chain. The protocol provides unlinkability between deposit and withdrawal addresses, not amount or balance confidentiality.

Anonymity — Unlinkability. Users can deposit assets into Privacy Pools and later withdraw them, either partially or fully, without creating an on-chain link between their deposit and withdrawal addresses. This breaks the linkability between the two interacting addresses so any external on-chain observer would not be able to see any association between the addresses.

Asset privacy — No. Users have to deposit their public tokens to the anonymity pool using a normal on-chain transaction that publicly shows which asset is being transferred. At the same time, withdrawal transactions send tokens from the smart contract to the recipient address.

Plausible deniability — No. Anyone can see that a user is interacting with the Privacy Pools entrypoint contract to deposit or withdraw funds. As a consequence, external on-chain observers can clearly see who is using the private transfer protocol and take specific actions (e.g. flag or ban addresses).

On-chain gas cost: transfer — 0. There is no private transfer inside the Privacy Pools v1 protocol. Users are only able to deposit funds using one address and withdrawing those funds to another unlinked address. The Privacy Pools v2 protocol would allow internal private transfers using a UTXO note-based architecture.

Time-to-finality — N/A. Privacy Pools is deployed on Ethereum mainnet. Finality is inherited from the underlying chains.

Number of secrets — 2. Each deposit generates two random values locally: a nullifier and a secret. These are Poseidon-hashed to produce a precommitment, which is combined with the deposit value and a label to form the on-chain commitment. The nullifier and secret are randomly generated, not derived from the wallet — they are an independent secret that must be backed up separately. The wallet key is the first secret, and the deposit note (nullifier + secret) is the second.

Deposit time — 0. Deposits can be made immediately with no pre-deposit waiting period enforced by the protocol. It is worth pointing out that Privacy Pools only allow for deposit and withdraw so even though there is no wait period in the deposit step, users will have to wait the withdraw time period in order to use the protocol with all its privacy features.

Withdraw time — 28800. After depositing, the Association Set Provider (ASP) must vet the deposit before a private withdrawal is possible. This vetting process can take up to 8 hours. If the ASP rejects the deposit or does not process it in time, users can recover funds publicly via the ragequit mechanism, which withdraws funds to the original depositing address. This action is public and is routed back to the depositing address, therefore there is no unlinkability unlike a regular withdrawal.

Censorship resistance — No. The Association Set Provider (ASP) is a closed framework at the moment and external ASPs cannot be configured in the current Privacy Pools contracts. In theory anyone could deploy new contracts with loose ASP controls but that would not be related to the official Privacy Pools protocol. The ASP is the core feature of the protocol because it allows users to maintain anonymity without mixing funds with malicious or sanctioned actors.

External network dependence — Yes, permissioned. The Association Set Provider needs to be running in order to allow deposit and withdraw function executions in the smart contracts. Hence the protocol depends on a permissioned network that authorizes smart contract interactions. If the ASP is taken down then the deposit and withdraw actions will be blocked. As stated in the censorship resistance property, there could be Privacy Pools contracts that do not rely on any ASP or rely on more trustless ASP.

Escape hatch — Instantly. Ragequit is a safety mechanism that allows the original depositor to publicly withdraw their funds without needing ASP approval. This operation serves as a critical backup withdrawal method which ensures the ability to withdraw user funds when the label is not approved by the ASP or its approval was revoked

Upgradeability — Multi-sig. The Privaye Pools Entrypoint contract uses the UUPS (Universal Upgradeable Proxy Standard) pattern with OpenZeppelin AccessControl. The owner role controls pool registration, configuration, and contract upgrades.

Client-side proving — Yes. A zk proof is required to withdraw previously deposited funds. The user is required to construct the withdrawal parameters and later generates a Groth16 proof locally of commitment ownership. The proof is verified on-chain by the smart contract.

Third-party inspectability — No. No third party can view the private information of users unless the users explicitly share their note secret values. The ASP compliance mechanism checks that the transaction is not coming from suspicious source but it never grants access to unauthorized third parties to see deposit and withdraw relations.

Implementation maturity — 4 : Mainnet for more than 1 year. The Privacy Pools deposit contract was deployed on March 29th 2025

Post-quantum secure — No. Privacy Pools uses Groth16 zk-SNARKs over the BN254 curve with Poseidon hashing. BN254 is based on elliptic curve cryptography, which is vulnerable to Shor's algorithm and could be attacked by quantum computers.

Layer of enforcement — Protocol/chain. The Association Set Provider blocks any suspicious withdrawal directly on the smart contracts. The user can still rage quit and withdraw to their original deposit address

Enforcement entities — Third party. The Association Set Provider (ASP) administrators. There is no open-source documentation explaining how the ASP monitors and flags transactions.

Type of compliance — Proof of innocence (POI) / ASP. The ASP vets deposits by checking the source of funds against illicit activity before adding them to the approved association set. This functions as a proof-of-innocence mechanism where only vetted deposits can be privately withdrawn. Depending on the ASP, there could be multiple compliance mechanisms used in order to valid if a transaction is safe or not. For example one ASP might check the transaction and funds history (Know Your Transaction KYT), another ASP might verify that the interacting address does not belong to a sanctioned list (Proof of Innocence).

Point of enforcement — Withdrawal. Withdrawals to new or unrelated addresses require a valid ASP root in the zk proof. Users need to generate this proof locally and the smart contract verifies it against the on-chain ASP root. If the proof cannot demonstrate label inclusion using the latest root, a user can use the ragequit mechanism to withdraw funds to the original depositing address, rather than an unlinked address.

Selective disclosure: viewing entity — User. Privacy Pools does not provide a mechanism for selective disclosure, compliance is handled by the association set provider mechanism. In theory, users could generate data that links deposit and withdraw actions. There is no tool or feature at the UI to create such a report, but it is technically possible.

Selective disclosure: viewing control — None. There is no viewing key system in the protocol. Privacy Pools is a mixer which provides unlinkability, so structured viewing permissions do not apply. A user could manually link their deposit and withdrawal using their secret note.

Cryptographic verifiability — Yes. Privacy Pools uses Groth16 zk-SNARKs over the BN254 curve, with Circom circuits and Poseidon hashing for commitments and nullifiers. Proofs are verified on-chain.

Open source — Yes. The core contracts (privacy-pools-core) are licensed under Apache 2.0. Website repos use other licenses. The ASP code, which controls deposit vetting and withdrawal eligibility, is not publicly available in the 0xbow GitHub organization.

Private State Scalability — Infinity grow. When a user deposits funds, a new commitment is registered in the smart contract. All commitments are inserted in a Lean Incremental Merkle Tree (LeanIMT), which grows indefinitely as new deposits are made. Withdrawal function calls add nullifier hashes to the smart contract to prevent double-spending. These nullifiers must be stored permanently to mark which commitments have been spent.

Client-side indexing — No scanning. Users withdraw funds by providing their secret note directly, without needing to scan the blockchain. The official UI displays the user's deposits and amounts, but this is based on locally stored note data rather than scanning.

Private state model — UTXO-based state model. Each deposit creates a unique commitment (analogous to a UTXO) that can be spent via withdrawal. To determine their total balance, a user sums the values of all their unspent commitments. Partial withdrawals create a new commitment for the remaining value.

Private Data Storage — Smart contracts. Each commitment (deposit receipt) is saved in an on-chain Lean Incremental Merkle tree (LeanIMT). The specific merkle tree was chosen due to its dynamic updates, storage and hashing optimizations.

Access to DeFi — No access to DeFi. Privacy Pools implemented a yield earning module for some of its token pools. This means that some tokens can earn yield privately while being idle on the anonymity pool. There is no mention of the yield module in the official docs but there is an announcement on social media and the UI shows some pools' APR percentage.

Programmability / Generality — Only payments. Privacy Pools v1 is similar to Tornado Cash. The main goal is to break linkability between deposit and withdraw function executions. Privacy Pools v2 is aiming to introduce private transfers inside the shielded pool. There are no plans to build programmability or generality on top of Privacy Pools in the short and medium term.

Railgun

A privacy system for Ethereum using zk-SNARKs to shield ERC-20 tokens and ETH inside a smart contract, enabling private transfers and shielded DeFi interactions.

Categories: Shielded pool, Zero Knowledge Proofs (ZKPs)

Documentation: https://docs.railgun.org/wiki/learn/privacy-system

Confidentiality — Yes. RAILGUN private (shielded) balances are built through a registry of UTXOs (unspent transaction outputs), similar to Bitcoin. The primary difference is that the RAILGUN UTXO list is encrypted with the sender and receiver's keys, meaning that no outsider can view the contents of the UTXO unless it is the recipient of that transaction. In order to send a transaction, the user consumes their available UTXO notes and creates new encrypted notes that are sent on-chain in order for the receiver wallet to find them and add them to the aggregated user's balance. These UTXOs form a Merkle tree — a structured hash list used to validate the balance state during transactions. To reconstruct a wallet's private balance, each commitment leaf in the tree must be synced and decrypted, a process that takes a few minutes before UTXO token balances are aggregated.

Anonymity — Yes. When a user sends a transaction using Railgun, they consume their available UTXOs (notes) by adding a nullifier to the on-chain state. In the same step new UTXOs belonging to the recipient are generated and added to the commitments merkle tree. These new UTXOs are encrypted and only the recipient has the ability to check their values and aggregate their total balance. An outside on-chain observer can see commitments added (notes generated) and nullifiers (notes consumed) but cannot link the sender or recipient with a specific transaction.

Asset privacy — Yes. Everybody can see ERC-20 tokens interact with the Railgun contract in the deposit and withdrawal steps. Therefore there is no asset privacy when users are entering or leaving the Railgun protocol. But once inside the pool, it is not possible to identify which asset is being transferred because the on-chain actions are only notes creation (commitments added) and notes consumption (nullifiers added).

Plausible deniability — No. All the 3 steps (shield, transfer or unshield) require an interaction with the Railgun smart contracts. These actions publicly show that an address is interacting with the Railgun protocol. A user can use a community broadcaster to submit encrypted transaction information on-chain when sending private transfers. This method could potentially allow the user to plausible deny sending a specific transaction inside the Railgun protocol.

Time-to-finality — N/A. RAILGUN is deployed on Ethereum mainnet (as well as BSC, Polygon, and Arbitrum). Time to finality is based on the underlying blockchain finality.

Number of secrets — 1. Users store one mnemonic seed phrase. The spending key (Baby Jubjub curve, BIP-32 derivation) and viewing key (Ed25519/EdDSA) are both deterministically derived from this seed. A database encryption key protects the stored mnemonic but is derived from a user-provided password, not an independent secret.

Deposit time — 3600. Users depositing funds (shielding funds) into the Railgun smart contracts have to wait a time period for List Providers to check that the transaction origin is clean. According to the Railgun documentation, the time period could take around an hour for most users. In this standby period the user could withdraw their funds to the depositing address at any time.

Withdraw time — 0. No mandatory withdrawal waiting period enforced by the protocol. Unshielding (withdrawing) is available as soon as the user constructs the required ZK proof to withdraw. There is no standby wait period because the withdraw transaction only checks that the Private Proof of Innocence (PPOI) proofs of that transaction are valid.

Censorship resistance — Yes. Railgun is censorship resistant at the smart contract level — any user can interact with the contracts directly without restriction. However, the practical user experience is constrained by wallet providers and community broadcasters that require valid Private Proofs of Innocence (POI) proofs. Users who are flagged by list providers may have fewer options for submitting transactions through standard wallets, though direct contract interaction remains available.

External network dependence — No. RAILGUN enables encrypted private transactions through Shielding. Once assets are shielded into the RAILGUN contract, there are several options: transfer tokens, use a cross-contract call, unshield tokens. In order to achieve full anonymity, RAILGUN uses a network of Broadcasters to submit transactions. This network is not obligatory, a user can directly interact with the smart contracts

Escape hatch — Can exit in a time period. With Private Proofs of Innocence, there is an Unshield-Only Standby Period, during which the only available action will be an unshield interaction to return tokens back to the original wallet. This Unshield-Only Standby Period is to give List Providers enough time to update their data such that bad actors are unable to hop addresses quickly to outrun data updates. As always, during this Unshield-Only Standby Period, users retain full control and custody over their tokens and can be unshielded back to the shielding address if needed urgently.

Upgradeability — DAO. RAILGUN governance is decentralized, any wallet that locks RAIL can vote on and submit proposals to vote. For all governance stages, 1 RAIL locked is equal to 1 vote, snapshots of voting power are taken at each governance stage. Any change to the smart contracts (including the governance, treasury, and voting contracts) requires the passing of an on-chain proposal. The decentralized governance system is the only way in which upgrades to the RAILGUN protocol can be made.

Client-side proving — Yes. There is a distinct Groth16 prover for node/browser (snarkjs) and a different prover for mobile (native prover). Both provers generate Groth16 snarks using different assembly processes that are compatible with different platforms.

Third-party inspectability — No. No third party can inspect private transaction data within Railgun. Shielded transactions are encrypted such that only the sender and recipient can decrypt the contents. Broadcasters relay encrypted transaction data on-chain but cannot decrypt or modify it — any alteration would invalidate the cryptographic proof and be rejected by the smart contracts.

Implementation maturity — 5 : Mainnet for more than 2 years. Railgun deployed on Ethereum mainnet in July 2021, making it operational for over 4 years. It has since expanded to BSC, Polygon, and Arbitrum.

Post-quantum secure — No. RAILGUN's zk-SNARK circuits are proved using the Groth16 proof system, a pairing-based zk-SNARK design. Groth16 is based on Elliptic Curves Cryptography (ECC)

Layer of enforcement — App. Private Proofs of Innocence (POI) is enforced at the wallet and broadcaster layer, not at the smart contract level. The RAILGUN smart contracts themselves do not reject transactions based on POI status. Instead, wallet providers require valid POI proofs before constructing transactions, and community broadcasters will not relay transactions without valid proofs. Users retain the ability to interact with the contracts directly, bypassing POI enforcement.

Enforcement entities — Third party. Private Proofs of Innocence List Providers: Elliptic ScamSniffer PureFi SlowMist Chainalysis Sanctions Oracle

Type of compliance — Proof of innocence (POI) / ASP. Railgun implements Private Proofs of Innocence (POI), where users generate a recursive zk-SNARK proving their shielded tokens are not associated with addresses on blocklists maintained by list providers. Proofs are generated locally by the user after the initial shield and carried forward for every subsequent transaction.

Point of enforcement — Deposit. Private Proofs of Innocence checks begin when tokens enter the RAILGUN privacy system after a shield interaction.

Selective disclosure: viewing entity — User, Voluntary third-party disclosure. Viewing Keys enable the holder to decrypt encoded interaction information but not send interactions. They scan the RAILGUN smart contract events to reveal what has been sent to users and what interactions

users have sent. Users can share viewing keys with third parties for auditability.

Selective disclosure: viewing control — Pre-defined. Viewing keys currently grant full read access to all transactions for a given address — they cannot be scoped or restricted. An upcoming update will allow viewing keys to be scoped by block number (e.g. only showing transactions from block X to block Y), which would make this programmable. Until that update ships, viewing control is pre-defined.

Cryptographic verifiability — Yes. Railgun uses the Groth16 proof system on the BN254 curve to verify shielded transactions on-chain. Spending keys use the Baby Jubjub elliptic curve with BIP-32 derivation, viewing keys use Ed25519 (EdDSA), and note commitments use Poseidon hashing. All state transitions are cryptographically verified by the smart contracts without revealing transaction details.

Open source — No. Railgun's smart contracts are open source under the Unlicense (public domain) and the wallet SDK and engine use the MIT license. However, the core ZK circuits (circuits-v2 and circuits-ppoi) are source-available but not open source — circuits-v2 explicitly states 'No License is provided' and circuits-ppoi lists 'NONE'. The circuits can be inspected on GitHub but cannot be legally copied, modified, or redistributed.

Private State Scalability — Infinity grow. The RAILGUN system contains an efficient internal implementation of a batch-incremental Merkle tree. Each shielded interaction generates a new note which takes the form of a new root or leaf on the Merkle tree. All tokens within the RAILGUN system share the same Merkle tree. Every interaction that updates the state of the Merkle tree will generate a new Merkle root or leaf that is hashed and secured by the zk-SNARK circuits.

Client-side indexing — Partial scanning. While generating the most up-to-date merkle tree, each commitment leaf (which represents a transaction) must be synced and decrypted in order to build a RAILGUN wallet’s private balance. This syncing process can take a few minutes. Once the sync is complete, UTXO token balances are aggregated into a user’s balance.

Private state model — UTXO-based state model. RAILGUN operates on an encrypted UTXO model. Transaction outputs are completely secure from outside observers. Each UTXO is an encrypted note of a public key that establishes who can spend the underlying asset, amount, token ID, and a randomness field.

Private Data Storage — Smart contracts. All UTXO commitments, nullifiers, and Merkle tree roots are stored in Railgun's on-chain smart contracts on Ethereum and supported chains. Encrypted note data is emitted as events and persisted on-chain, allowing wallets to reconstruct private balances by scanning and decrypting commitment leaves.

Access to DeFi — Unlimited access to DeFi applications. Railgun provides unlimited DeFi access through its relay adapt contract, which can execute arbitrary external contract calls against shielded balances. An unshield moves tokens temporarily into the relay adapt contract, a multi-

call executes any number of contract interactions, and the result is shielded back into the user's private balance — all in a single atomic transaction.

Programmability / Generality — Partial programmability. Cross Contract calls are the primary method for RAILGUN to access external DeFi protocols and smart contracts. Any number of ordered contract calls can be wired together and executed against a private balance. These cross-contract interactions occur in the generalized RAILGUN Relay Adapt contract, which facilitates a number of serial multi-calls in a single block.

Redact

A confidential transfer protocol on Ethereum using Fhenix's CoFHE (Coprocessor for Fully Homomorphic Encryption) to encrypt ERC20 token balances on-chain. All FHE computations are handled off-chain by the coprocessor network. Balances are hidden while addresses are fully visible.

Categories: Homomorphic encryption (FHE - HE)

Documentation: https://docs.redact.money/

Confidentiality — Yes. Redact uses Fhenix's coprocessor to encrypt ERC20 token balances on-chain. Balances are stored as ciphertexts and transactions modify them using FHE operations. Nobody except the owner can see the plaintext balance or the transaction amounts.

Anonymity — No. Redact operates on Ethereum where all transaction senders and recipients are publicly visible on-chain. The protocol encrypts only token amounts and balances, not the addresses involved in transfers.

Asset privacy — No. The asset type is directly inferred by the contract under a token model, like ERC20. During the encryption step, users approve the specific token contract, which is a public on-chain operation. Therefore the contract address is visible to any observer, revealing which asset type is being used.

Plausible deniability — No. The encryption operation is a direct call to the Redact smart contract, making it publicly visible on-chain that the user is interacting with the confidential transfer protocol. Both sender and receiver encrypted balances are updated on each transfer. External observers can see that two Redact accounts have been updated.

Time-to-finality — N/A. Redact is deployed on Ethereum mainnet. Finality is inherited from the underlying chain. The roadmap mentions future expansion to additional blockchains.

Number of secrets — 1. Users only need their existing Ethereum wallet to interact with Redact. Permits are temporary EIP-712 signatures derived from the wallet key with a 24-hour default expiration. Balances are encrypted using the CoFHE network's TFHE public key, which is published on-chain. The corresponding private key is distributed across threshold network nodes using MPC, so no single node holds the decryption key.

Deposit time — 0. No mandatory deposit waiting period enforced by the protocol. Encryption requires two transactions: the first approves and encrypts the ERC20 tokens via the CoFHE coprocessor, which introduces some operational latency for FHE computation. This is not a protocol-enforced delay.

Withdraw time — 0. No mandatory withdrawal waiting period enforced by the protocol. Decryption requires two separate transactions: the first triggers an on-chain decryption request via the CoFHE coprocessor, which typically takes 5-10 seconds to finalize. The second transaction claims the decrypted ERC20 tokens back.

Censorship resistance — No. Redact depends on Fhenix's CoFHE coprocessor network for all FHE operations. The threshold network is permissioned with predefined roles (coordinator, party members, trusted dealer) and no public process for joining as a node operator. The network runs an MPC protocol to decrypt ciphertexts. Full ciphertexts are stored on an off-chain data availability (DA) layer, with only 128-bit handles stored on-chain. The coprocessor could refuse to process operations for specific users or transactions.

External network dependence — Yes, permissioned. Redact relies on Fhenix's CoFHE coprocessor network for all FHE computations. The architecture consists of a task manager contract, an aggregator, the fheOS computation engine, and a threshold network for decryption via MPC. Full ciphertexts are stored on an off-chain DA layer with only 128-bit handles on-chain. A trusted dealer initializes the threshold network's key distribution (with plans to eliminate this dependency). If the coprocessor network goes down, the protocol cannot process transfers or decrypt balances.

Escape hatch — No. There is no escape hatch mechanism for encrypted balances. Decryption requires the CoFHE threshold network's MPC protocol. If the coprocessor becomes unavailable or censors a user, there is no alternative cryptographic path to recover encrypted funds. Once a user has completed both decryption transactions and claimed their ERC20 tokens, those funds are standard tokens and no longer depend on CoFHE.

Upgradeability — Network upgrade (hard/soft fork). The fhERC20 smart contracts do not document on-chain upgradeability. However, the CoFHE coprocessor network (threshold network, fheOS, DA layer) is separately managed by Fhenix and can be upgraded independently, making the system as a whole upgradeable even if the on-chain contracts are immutable.

Client-side proving — N/A. Redact executes FHE computation via an off-chain coprocessor network, no client-side proving is needed. All cryptographic operations are processed by the external CoFHE network and do not involve proof generation on the user's device.

Third-party inspectability — Yes. The threshold decryption network, operated by Fhenix, holds distributed shares of the FHE private key and uses an MPC protocol for collaborative decryption. A colluding majority of party members could reconstruct the key and decrypt user balances. The system currently uses a trusted dealer to initialize secret shares, with plans to eliminate this centralization risk.

Implementation maturity — 2 : Public testnet. Redact is currently in Phase 0 (testnet deployment) per its roadmap. The protocol is available on public testnet with core encrypt, decrypt, and confidential transfer functionality operational.

Post-quantum secure — No. The TFHE encryption layer uses lattice-based cryptography (LWE problem), which is believed to be post-quantum secure. However, the underlying Ethereum transactions use ECDSA signatures, and the threshold network and DA layer likely use classical cryptographic primitives that are not post-quantum resistant. The system as a whole is not post-quantum secure.

Layer of enforcement — None. Redact currently has no built-in compliance enforcement mechanism. The protocol focuses solely on confidential transfers without restricting who can participate.

Enforcement entities — None. There is no compliance enforcement entity in Redact. The coprocessor network could collude with an enforcement entity to decrypt specific balances or censor specific addresses.

Type of compliance — Selective disclosure. No enforced compliance mechanism exists. Users can issue Fhenix Permits, which are signed EIP-712 messages that authorize off-chain decrypt and seal operations on encrypted balances. Permits come in three types (self, sharing, and recipient), are network-specific, expire after 24 hours by default, and require signature verification. This enables voluntary selective disclosure to authorized parties. Selective disclosure of transaction history and DeFi actions will be possible in the future.

Point of enforcement — None. There is no compliance enforcement at any point in the transfer lifecycle. Anyone can encrypt, transfer, and decrypt tokens without restriction.

Selective disclosure: viewing entity — User, Voluntary third-party disclosure. Users can issue EIP-712 permits that grant temporary, scoped read access to their encrypted balances. These permits allow the user to voluntarily disclose their confidential balance to any authorized smart contract or third party.

Selective disclosure: viewing control — Pre-defined. Permits use a fixed EIP-712 schema with three defined types (self, sharing, recipient) and a 24-hour default expiry. Users can issue permits within this fixed structure but cannot create custom viewing policies or modify the disclosure schema. Selective disclosure of transaction history and DeFi actions will be possible in the future.

Cryptographic verifiability — No. Correctness rests on a permissioned threshold-decryption network (an economic / multi-party trust mechanism), so the value is No per the rule that excludes economic mechanisms from cryptographic verifiability. The CoFHE architecture claims to use zero-knowledge proofs of knowledge (ZKPoK) alongside TFHE, but even if ZKPoK is implemented, decryption itself depends on threshold-network honesty rather than pure math.

Open source — No. The Redact repository and cofhe-contracts are licensed under MIT. However, the fheOS computation engine, threshold network node code, and DA layer do not appear in the FhenixProtocol GitHub organization. These are critical infrastructure components.

Private State Scalability — Infinity grow. Redact uses an account-based model where each user's encrypted balance handle is stored as a mapping within the smart contract storage. On-chain state grows proportionally to the number of accounts. Full ciphertexts grow on the off-chain DA layer proportionally to the number of operations — every transfer emits a new ciphertext that must be retained for balance reconstruction.

Client-side indexing — No scanning. Since Redact uses an account-based model with encrypted balances stored directly as contract state, users can retrieve their encrypted balance directly from the contract without scanning past blockchain events.

Private state model — Account-based state model. Each Redact account holds an encrypted balance stored directly in the smart contract. The balance ciphertext is updated in-place on each transfer

Private Data Storage — Smart contracts. Ciphertext handles (128-bit pointers) are stored in the Redact smart contract state as account balance mappings. Full ciphertexts are too large for on-chain storage and are kept on an off-chain data availability (DA) layer.

Access to DeFi — No access to DeFi. The fhERC20 standard is designed for compatibility with existing wallets and tooling. The docs describe Redact as the foundation for private DeFi with plans for lending, borrowing, and swapping. Currently only confidential transfers are supported, so there is no live DeFi access.

Programmability / Generality — Partial programmability. The CoFHE framework enables smart contracts to process encrypted data via FHE operations (addition, subtraction, conditional logic on ciphertexts). Redact currently implements only confidential ERC20 transfers, but the underlying framework supports more general encrypted computation. The docs position Redact as the foundation for a private DeFi ecosystem.

Tongo

A confidential payment protocol on Starknet that uses ElGamal homomorphic encryption and zero-knowledge proofs to enable private ERC20 token transfers. Account balances are stored as encrypted ciphertexts on-chain, hiding amounts while keeping the protocol verifiable and supporting an optional auditor for regulatory compliance.

Categories: Homomorphic encryption (FHE - HE)

Documentation: https://docs.tongo.cash/

Confidentiality — Yes. Tongo uses ElGamal homomorphic encryption over the Stark curve to store account balances as encrypted ciphertexts. Each account maintains a current balance (spendable) and a pending balance (received but requiring an explicit rollover operation before

spending). The pending balance exists to prevent balance manipulation attacks where an attacker sends a small encrypted amount to a sender mid-proof-generation, changing the sender's balance and invalidating their ZK proof. Both balances are stored as ciphertexts invisible to external observers. Mathematical operations (additions, subtractions) are performed homomorphically without decryption.

Anonymity — No. Tongo uses an account-based model where each account is identified by a public key. Transfers are sent from one account's public key to another, meaning both the sender and receiver addresses are visible on-chain when a Tongo transaction is submitted. The protocol provides no anonymity mechanism.

Asset privacy — No. When funding a Tongo account, the user deposits a specific ERC20 token into the contract. When withdrawing, the smart contract sends the specific ERC20 token to the user. When transacting between Tongo accounts, the transaction shows the ERC20 token being sent but the amount and balances are encrypted. The asset type is visible in all three operations: funding, withdrawing, and transacting.

Plausible deniability — No. Tongo interactions (funding, withdrawing, and transacting) are direct calls to the Tongo smart contract, making it publicly visible that users are interacting with the privacy protocol. There is no way a user can plausibly deny using the protocol or sending tokens to a Tongo account.

Time-to-finality — N/A. Tongo is deployed on the Starknet L2, a ZK-rollup on Ethereum. Finality is inherited from the underlying chain. Starknet's time to L1 finality at the time of writing is around 29 minutes. Finality is dependent on the sequencer submitting proofs to the L1. After proof submissions, SHARP (StarkWare's proof aggregator) consolidates all proofs into a single STARK proof that gets posted and verified on Ethereum L1.

Number of secrets — 2. Users require a Starknet wallet to submit on-chain transactions and a separate Tongo ElGamal keypair over the Stark curve for encrypting and decrypting private balances. The Tongo private key is independently generated, not derived from the Starknet wallet.

Deposit time — 0. No mandatory deposit waiting period. Funding a Tongo account is an immediate Starknet transaction.

Withdraw time — 0. There is no mandatory withdrawal waiting period. Users can withdraw their partial or full balance at any time, including via the instant ragequit mechanism (withdraw all current balance to a public Starknet account address).

Censorship resistance — Yes. Tongo supports multiple optional compliance models configured at deployment: a global auditor for real-time transaction monitoring, selective disclosure via viewing keys, ex-post proving for retroactive investigations, range compliance (transfer amount thresholds), velocity limits (cumulative amount bounds), and whitelist compliance (authorized recipient addresses). The auditor can examine but not censor transactions.

External network dependence — No. Tongo requires no trusted setup ceremony. The protocol's cryptography relies solely on the discrete logarithm assumption over the Stark curve, eliminating the need for any external trusted parties or multi-party computation ceremonies. All operations are self-contained within the Starknet smart contracts.

Escape hatch — Instantly. Tongo provides a ragequit mechanism that allows users to instantly withdraw their full current balance back to ERC20 tokens. The user submits a ZK proof demonstrating ownership of the account private key and that the disclosed amount equals their full balance. The contract converts the encrypted balance to ERC20 and sends it to the specified address. The pending balance is not affected by this operation.

Upgradeability — Single admin. The deployed contracts show OpenZeppelin Cairo access control functions (is_upgrade_governor, has_role, get_role_admin), indicating the contracts use Starknet's native upgradeability via the replace_class syscall with role-based access control. The contract owner can also rotate the auditor public key.

Client-side proving — Yes. Tongo uses ElGamal homomorphic encryption and Sigma protocol zero-knowledge proofs over the Stark curve to update on-chain encrypted balances. The SHE (Somewhat Homomorphic Encryption) library provides proof primitives for operations like same-encryption verification, range proofs, and bit proofs. The SDK runs these provers on the user's device with no external proving service.

Third-party inspectability — Yes. If configured at deployment, a global auditor can decrypt balances and transfer amounts for regulation and compliance purposes. This is an optional feature - instances deployed without an auditor have no third-party inspectability. The auditor key can be rotated by the contract owner but cannot be added after deployment. The global auditor keys can be distributed across multiple parties using threshold encryption.

Implementation maturity — 3 : Mainnet for less than 1 year. Tongo has 7 mainnet instances deployed on Starknet (STRK, ETH, wBTC, USDC.e, USDC, USDT, DAI).

Post-quantum secure — No. Tongo's security relies on ElGamal encryption over the Stark curve, which is vulnerable to Shor's algorithm. Additionally, encrypted balances stored on-chain are subject to harvest-now-decrypt-later attacks, where adversaries collect ciphertexts today for future decryption by quantum computers.

Layer of enforcement — App. Compliance is enforced at the application layer through optional methods activated at deployment time: the global auditor mechanism that allows viewing access to authorities, range compliance that proves transfer amounts are below a threshold, velocity limits that prove cumulative amounts are within bounds, and whitelist compliance that proves a recipient is in an authorized addresses list.

Enforcement entities — Admin. The Tongo contract owner configures compliance features at deployment. All compliance mechanisms (global auditor, range compliance, velocity limits, whitelist) are optional and instance-specific. The auditor role is a designated party (single key or

threshold key) that can examine transactions.

Type of compliance — Programmatic policies, Selective disclosure. All compliance features are optional and configured per-instance at deployment. The global auditor enables selective disclosure of encrypted balances and transfer amounts. Advanced features include range compliance (transfer amounts below a threshold), velocity limits (cumulative amounts within bounds), and whitelist compliance (authorized recipient addresses). These are programmatic policies proven via zero-knowledge proofs without revealing amounts.

Point of enforcement — Deposit, Transfer, Withdrawal. If configured, the auditor can examine balances or amounts at all stages of the transaction lifecycle: deposits (funding), internal transfers, and withdrawals. The other advance compliance features (range compliance, velocity limits and whitelist compliance) can be implemented at any step of the lifecycle.

Selective disclosure: viewing entity — Involuntary third-party disclosure, User. If an auditor is configured, it can view transaction details without user consent (involuntary third-party disclosure). Users can also voluntarily disclose transaction details to any third party via ex-post proving, which creates a new encryption of the transfer amount for the third party's public key with a ZK proof of correctness. The auditor is optional and instance-specific.

Selective disclosure: viewing control — Programmable. Tongo supports multiple configurable compliance mechanisms: the global auditor is set at deployment and can be rotated by the owner, ex-post proving allows users to create on-demand disclosures for any third party, and advanced features (range compliance, velocity limits, whitelist) provide programmable policy enforcement. Viewing keys can be jurisdiction-specific with time-limited access.

Cryptographic verifiability — Yes. Tongo uses ElGamal homomorphic encryption over the Stark curve combined with Sigma protocol zero-knowledge proofs to ensure that all operations (fund, transfer, withdrawal) are cryptographically verifiable. The protocol relies on the discrete logarithm assumption over the Stark curve and requires no trusted setup.

Open source — Yes. The Tongo smart contracts (Cairo) and TypeScript SDK are licensed under Apache-2.0. The SHE cryptographic library used for Sigma protocol proofs is also Apache-2.0. Both are publicly available under the fatlabsxyz GitHub organization.

Private State Scalability — Stateless. Tongo uses an account-based model where each user's encrypted balance is stored as a mapping within the smart contract storage. New users add entries to the contract's account mapping, and the state grows proportionally to the number of accounts rather than to the number of transactions.

Client-side indexing — No scanning. Because Tongo uses an account-based model with balances stored directly as contract state, users can retrieve their encrypted balance directly from the contract without scanning past blockchain events or transactions.

Private state model — Account-based state model. Tongo maintains an account-based private state where each account holds an encrypted current balance (immediately spendable) and an encrypted pending balance (received but not yet rolled over). Users must explicitly rollover pending funds to make them spendable.

Private Data Storage — Smart contracts. All private state—including encrypted account balances (current and pending) and auditor configuration—is stored directly in the Tongo smart contract on Starknet.

Access to DeFi — Composable interface, but requires DeFi protocol changes. Tongo uses an ERC20-compatible encrypted token interface. The docs list AMMs (hidden trade sizes) and DAO governance (confidential voting) as use cases. DeFi protocols would need to be built or adapted to work with encrypted balances and ZK proofs. Currently no DeFi integrations exist, but the design is composable.

Programmability / Generality — Transfers and DeFi operations. Tongo currently implements confidential payment operations (fund, transfer, rollover, withdraw). The SHE library provides general-purpose ZK proof primitives that support building AMMs with hidden trade sizes and DAO governance with confidential voting, as described in the use cases documentation.

Tornado Cash

A non-custodial, Ethereum-based mixer that uses zk-SNARKs to break the on-chain link between deposit and withdrawal addresses for fixed-denomination ETH and ERC-20 deposits.

Categories: Mixer, Zero Knowledge Proofs (ZKPs)

Documentation: https://docs.tornadocash.eth.limo/

Confidentiality — No. Tornado Cash uses fixed-denomination pools, so deposit and withdrawal amounts are publicly visible on-chain (e.g. 0.1, 1, 10, or 100 ETH). The protocol provides unlinkability between deposit and withdrawal addresses, not amount or balance confidentiality.

Anonymity — Unlinkability. Tornado Cash achieves anonymity by breaking the on-chain link between source and destination addresses via mixing pools. These pools are immutable smart contracts that accept fixed denominations of a currency (e.g. 0.1 ETH, 1 ETH, 10 ETH, 100 ETH), acting as a mix network. A user deposits a specific amount of ETH with one wallet and withdraws that same amount with another wallet totally disconnected from the former one.

Asset privacy — No. The user sends a deposit transaction that sends their public assets to the Tornado Cash contracts. Everybody can see the asset type of that specific transaction. When the user withdraws the funds with a different address, the public assets are sent to the receiver address and the asset type is again visible. Tornado Cash's main feature is unlinkability, not asset privacy.

Plausible deniability — No. Everyone with access to on-chain data can see which addresses have interacted with the Tornado Cash pools and smart contracts. Depositors and withdrawers are identifiable by their interactions with the contract. There is no way to deny that a specific address has used Tornado Cash, as shielded transactions are distinguishable from regular transfers.

On-chain gas cost: transfer — 0. There are no internal transactions in classic Tornado Cash. A user could "transfer" their secret note to allow another user to withdraw their shielded tokens. This transfer would happen off-chain and cannot be considered a Tornado Cash private transfer because it is not described in the documentation and was not in the original design. The new version called Tornado Cash Nova allows shielded transfers similar to shielded pool protocols

Time-to-finality — N/A. Tornado Cash is deployed on Ethereum, BNB Chain, Polygon, Arbitrum, Optimism, Avalanche, and Gnosis. Finality is inherited from each underlying chain.

Number of secrets — 2. Each deposit generates a random 62-byte preimage (nullifier + secret), which is Pedersen-hashed on the Baby Jubjub curve to produce a commitment stored in a Merkle tree. The nullifier and secret are randomly generated in the user's browser, not derived from the wallet key. This deposit note is an independent secret that must be backed up separately. The wallet key is the first secret, and the deposit note is the second. The Note Account feature allows encrypted on-chain backup of notes using the wallet key, but the note itself is not derivable from the wallet.

Deposit time — 0. No mandatory deposit waiting period enforced by the protocol. Deposits are immediate on-chain transactions.

Withdraw time — 0. No mandatory withdrawal waiting period enforced by the protocol. Although waiting longer between deposit and withdrawal improves privacy by growing the anonymity set, this is a user recommendation, not a protocol constraint.

Censorship resistance — Yes. Tornado Cash smart contracts are immutable and permissionless. In August 2022, the US government sanctioned addresses that had interacted with Tornado Cash and multiple frontend applications were taken down. Transactions were censored at the block-building level by compliant MEV relays. Despite this, the smart contracts remained functional and users were able to interact with them directly or through alternative frontends. A relayer network exists for gas-free withdrawals but is optional — users can call the contracts directly.

External network dependence — No. A relayer network handles withdrawals so that the recipient address does not need gas funds — the relayer pays fees and deducts them from the withdrawn amount. However, relayers are optional: users can call the withdraw function on the smart contract directly if they are willing to fund gas from the recipient address. The protocol has no dependency on any external network with additional crypto-economic assumptions.

Escape hatch — Instantly. Users can withdraw their funds from Tornado Cash at any time directly through the smart contracts even if the whole relayers network is taken down. It is recommended to wait a time period to withdraw funds and follow some tips to remain anonymous using Tornado

Cash. An instant deposit and withdraw pattern could allow external observers to link the depositor address to the receiver address.

Upgradeability — Immutable. There is a decentralized governance structure that uses the TORN token as its voting token. It does not control the Tornado Cash Classic anonymity pools but it can control other parts of the protocol and its community tools (like the entrypoint contracts or the relayers network)

Client-side proving — Yes. Groth16 zk-SNARK proofs over the BN254 curve are generated on the user's device without any external service. The proofs demonstrate inclusion in the Merkle tree of a specific pool contract.

Third-party inspectability — No. No third-party can link deposit to withdraws or obtain total user balances. In order to do so, third parties would require all secret values to generate the private notes of a specific user. If the Note Account feature is enabled, this data is encrypted using a user's key and submitted on-chain. The user balance or linkability will be totally safe unless the user decides to share their private key to a third party.

Implementation maturity — 5 : Mainnet for more than 2 years. Tornado Cash Classic pool contracts have been live on Ethereum mainnet since December 2019. The Router contract was deployed in February 2022 as an additional entrypoint, but the core protocol has been operational for over five years.

Post-quantum secure — No. Tornado Cash uses Pedersen hashing and Groth16 zk-SNARKs over the BN254 curve. These elliptic curve-based primitives are vulnerable to Shor's algorithm and could be attacked with quantum computers in the future.

Layer of enforcement — None. There is no compliance enforcement layer. Anyone can set up their own UI and relayer to interact with the deployed smart contracts. This lack of enforcement motivated the 2022 sanctions by the US government.

Enforcement entities — None. External entities such as exchanges or dApps could potentially block access to addresses that have interacted with Tornado Cash. In practice this happens with centralized exchanges and AML-compliant dApps that use external enforcement strategies provided by third parties.

Type of compliance — Selective disclosure. There is no enforced compliance, but users can generate selective disclosure reports linking deposit and withdrawal to show to a regulator.

Point of enforcement — None. There is no compliance enforcement in Tornado Cash itself. External dApps or web apps (e.g. centralized exchanges) can apply their own enforcement rules to any address that have interacted with the protocol.

Selective disclosure: viewing entity — User. Users can generate their own reports linking deposit and withdraw in order to share it with other third parties. This feature allows for easier tax reporting and even in authorities involved investigations.

Selective disclosure: viewing control — Pre-defined. The compliance tool generates a static report linking a deposit to its withdrawal using the private note. Once shared, the report cannot be revoked or scoped. The user decides who to share it with, but the disclosure structure is fixed by the protocol.

Cryptographic verifiability — Yes. Tornado Cash uses Groth16 zk-SNARKs over the BN254 curve with Pedersen hash commitments stored in Merkle trees. The deposit commitment and withdrawal proof are verified on-chain by the smart contract's verifier, ensuring correctness without revealing the link between depositor and withdrawer.

Open source — Yes. The core smart contracts (tornado-core) are licensed under GPL-3.0. The UI and relayer are MIT licensed. The CLI is ISC licensed. All 47 repositories in the GitHub organization are archived following the 2022 OFAC sanctions but remain publicly accessible. The community maintains mirrors on independent git instances.

Private State Scalability — Infinity grow. Each deposit generates a commitment that is stored in a Merkle tree. As more transactions happen, more commitments are added as leaves. Every withdrawal transaction stores a nullifier hash on-chain to signal that a specific note has been spent. Both data structures grow with the number of transactions.

Client-side indexing — No scanning. You need to manually input your private note in order to be able to withdraw funds. In case of using a Note Account, the encrypted backup is stored in a smart contract storage so it is easily accessible without scanning past transactions

Private state model — UTXO-based state model. Each deposit acts as a private note that gives access to the specific amount of funds related to that deposit transaction. To find their total balance, a user sums all their unspent private notes.

Private Data Storage — Smart contracts. The Tornado Cash smart contracts store all required data for the protocol to work. Every commitment is saved in a Merkle tree within the specific pool contract for that denomination. Nullifier hashes are also stored on-chain to prevent double-spending. Encrypted deposit data is stored in the smart contracts when using the Note Account feature.

Access to DeFi — No access to DeFi. In order to interact with any DeFi protocol, the user needs to withdraw funds from the Tornado Cash contracts first. A user could use Tornado Cash in a larger workflow where they do not want their main address linked to a specific DeFi action they will perform later.

Programmability / Generality — Only payments. Tornado Cash Classic is a set of tools to break the link between deposits and withdrawals. Tornado Cash Nova allows shielded transfers between two users that have an account inside the Tornado Cash ecosystem. There is no programmable logic beyond fixed-denomination deposits, withdrawals, and Nova shielded transfers.

WORM

WORM is a protocol that uses an ERC-20 token (BETH for Burned ETH) which is minted via the process of burning Ethereum (ETH) by sending it to unclaimable addresses. Users generate a proof-of-burn receipt which enables the minting of BETH. ETH transfers sent to an unclaimable address are indistinguishable from a regular ETH transfer to a fresh address. There is no way for an outside observer to detect whether a user is interacting with the WORM protocol. The protocol is based on EIP-7503, which standardizes Private Proof-of-Burn.

Categories: zkWormholes, Zero Knowledge Proofs (ZKPs)

Documentation: https://github.com/worm-privacy/whitepaper

Confidentiality — No. WORM and BETH are standard ERC-20 tokens with publicly visible balances and transfer amounts. The privacy mechanism only applies to the proof-of-burn process that breaks the link between burned ETH and minted tokens using zk-SNARKs. Once tokens are minted, all subsequent transfers, balances, and amounts are publicly visible on the Ethereum blockchain like any other ERC-20 token.

Anonymity — Unlinkability. WORM achieves unlinkability between sender and receiver. The burn-address is derived from a Poseidon hash of the burnKey, revealAmount, and burnExtraCommitment, truncated to 160 bits to form an Ethereum address. A nullifier prevents reuse of the same burn address. The zk-SNARK proof allows minting BETH tokens to any receiver address without revealing which burn-address the ETH came from, breaking the on-chain link between the original sender and the final receiver.

Asset privacy — No. WORM is a two-token protocol consisting of BETH (an ERC-20 proof-of-burn receipt) and WORM (a scarce ERC-20 minted from BETH). BETH is minted when ETH is burned, with each unit representing 1 ETH provably destroyed, while WORM is minted at 50 tokens per 30-minute epoch through BETH redemption. Since both tokens are standard ERC-20s on Ethereum, observers can distinguish which asset (BETH or WORM) is being transferred in any transaction, meaning asset privacy is not provided.

Plausible deniability — Yes. Deniability applies to the deposit side, which is a native ETH transfer to an address derived from Truncate160(Poseidon4(prefix, burnKey, revealAmount, burnExtraCommitment)) — indistinguishable on-chain from an ordinary transfer to a fresh EOA. A burner does not need to interact with any WORM contract to destroy the ETH, so the burner is never forced to reveal participation. The recipient's mint transaction to the BETH verifier is observable, so deniability covers entry only, not withdrawal.

Time-to-finality — N/A. WORM is an ERC-20 token created through the destruction of Ethereum (ETH) in a privacy-preserving manner. As an application-layer smart contract protocol deployed on Ethereum mainnet, WORM inherits the finality characteristics of the underlying Ethereum mainnet network.

Number of secrets — 1. WORM requires users to store 1 secret: the burnKey, a randomly generated number that serves as the private key for the proof-of-burn protocol. The burn address is derived as the first 160 bits of Poseidon4(POSEIDON_BURN_ADDRESS_PREFIX, burnKey, revealAmount, burnExtraCommitment). The burnKey also derives the nullifier via Poseidon2(POSEIDON_NULLIFIER_PREFIX, burnKey) and coin commitments via Poseidon3(POSEIDON_COIN_PREFIX, burnKey, amount). Valid burnKeys must satisfy a proof-of-work constraint where Keccak(burnKey | revealAmount | burnExtraCommitment | 'EIP-7503') < THRESHOLD to increase bit-security.

Deposit time — 12. WORM requires burning ETH to an unspendable address via a standard Ethereum transfer, then generating a zk-SNARK proof to mint BETH. The burn transaction itself is a regular EOA-to-EOA transfer that confirms in approximately 12 seconds (one Ethereum block). No additional deposit time delay is enforced by the protocol beyond standard Ethereum block confirmation time.

Withdraw time — N/A. WORM operates as an application-layer protocol on Ethereum mainnet where users burn ETH to a burn address and then generate a zero-knowledge proof to mint BETH tokens. The protocol does not impose any time lock or waiting period between proof submission and token minting.

Censorship resistance — Yes. WORM allows users to burn ETH to derived one-time addresses, with ETH not held or controlled by any contract or intermediary. BETH is minted from a zk-SNARK proof, not a withdrawal action, and users interact directly with the deployed smart contracts on Ethereum mainnet to submit proofs and mint tokens. As an application-layer protocol on Ethereum, WORM inherits Ethereum's permissionless properties where any valid transaction can eventually be included by validators, with no relayers or intermediaries required for protocol interaction.

External network dependence — No. WORM operates entirely on Ethereum mainnet using deployed smart contracts without relying on any external network. Burns occur through standard Ethereum transactions, zk-SNARK proofs are generated off-chain using external provers due to the computational cost, and verification happens on-chain via the WORM smart contracts. No external validators, relayers, or separate consensus mechanisms are required beyond Ethereum itself.

Escape hatch — Instantly. WORM and BETH are ERC-20 tokens on Ethereum mainnet. Users can transfer these tokens using standard Ethereum transactions that rely only on Ethereum's consensus and cryptography, so in theory users can move their assets instantly. In practice the BETH/ETH DEX pools can have low liquidity and leave the user holding BETH waiting to swap it to ETH. At the current moment BETH and WORM utility depends on the WORM protocol working correctly. If the protocol shuts down, users will be holding tokens with limited utility until they swap it for native ETH

Upgradeability — Immutable. The BETH and WORM contracts deployed on Ethereum mainnet have no proxy pattern, admin controls, or governance system. Issuance rules (50 WORM per 30-minute epoch) and proof verification logic are encoded in the contracts and Circom circuits, with no on-chain mechanism to change them post-deployment.

Client-side proving — No. WORM circuits are zkSNARKs written in Circom for performance. The Ethereum Merkle Patricia Trie proofs to show a burn address received ETH require high computational resources, so WORM relies on off-chain proof generation using external provers. Therefore the protocol does not use client-side proof generation or browser-based proving. Users concerned about privacy compromise can setup their own provers in powerful-enough servers to generate their private proof-of-burn proofs.

Third-party inspectability — Yes. The UI uses external provers for generating private proof-of-burn proofs. The external provers can inspect the inputs and leak private information to third parties. Users worried about this can run their own prover in a powerful server to generate their proofs. This is only recommended for advanced technical users due to complexity

Implementation maturity — 3 : Mainnet for less than 1 year. BETH and WORM contracts were deployed on Ethereum mainnet in February 2026 (BETH ~Feb 12, WORM ~Feb 21), giving the protocol about two months of production history at the time of evaluation.

Post-quantum secure — No. WORM is not post-quantum secure. The protocol uses Groth16 zk-SNARKs for proof-of-burn verification, which rely on elliptic curve pairings over the BN254 curve. These pairing-based constructions depend on the discrete logarithm problem, which Shor's algorithm can solve efficiently on a sufficiently large quantum computer. While the protocol's hash functions (Poseidon2 and Poseidon4) have some resistance to quantum attacks as symmetric primitives, the core cryptographic proof system that validates ETH burns and enables BETH/WORM minting is vulnerable to quantum adversaries.

Layer of enforcement — None. WORM has no compliance enforcement. The smart contracts (BETH.sol and WORM.sol) and zk-SNARK circuits only enforce protocol correctness — burn address inclusion in Ethereum state, nullifier uniqueness, and per-epoch issuance caps — not any compliance policy such as blocklists, sanctions screening, or identity checks.

Enforcement entities — None. No entity can blocklist addresses, freeze transfers, or modify protocol parameters. The contracts have no admin keys, governance tokens, or upgrade mechanisms, and the protocol operates permissionlessly without any DAO or third-party compliance provider.

Type of compliance — None. No identity verification, transaction screening, proof-of-innocence requirement, selective disclosure, or programmatic policy is enforced. The burn-and-mint flow using Groth16 zk-SNARKs provides privacy through cryptographic proofs but imposes no compliance checks.

Point of enforcement — None. No compliance is enforced at deposit, transfer, or withdrawal. The burn, mint, and ERC-20 transfer paths validate only cryptographic and protocol invariants (proof validity, nullifier uniqueness, per-epoch issuance caps), not any compliance policy.

Selective disclosure: viewing entity — User. The privacy model relies on unlinkability between burn addresses and minted BETH/WORM tokens through zk proofs, not on encrypted transaction data with optional disclosure. Users could voluntarily disclose their burnKey to reveal the burn address and burn amount after they have mint the desired tokens. Anyone with the burnKey is able to see the transaction history and also mint pending tokens. The burnKey can be considered a spending key until the tokens are minted and a viewing key for historical data. User would need to use a different burnKey in the future to avoid unauthorized spending.

Selective disclosure: viewing control — None. No formal selective disclosure mechanism exists. The user could share his burnKey after minting the tokens to allow a third party access to historical burn data. It is worth pointing out that sharing the burnKey is similar to sharing a private key so the user will have to use a different burnKey in the future to avoid unathorized spending.

Cryptographic verifiability — Yes. WORM uses zk-SNARK circuits to cryptographically verify that users burned ETH to a specific address without revealing which address, by proving Merkle-Patricia-Trie inclusion of the burn address in Ethereum's state root and checking that the hash of each trie layer is contained in the next layer. The burn address is derived from a burn-key using Poseidon hashing, with an additional Keccak256 proof-of-work constraint requiring the hash to begin with two zero bytes, and a nullifier prevents reuse of the same burn-key.

Open source — Yes. The proof-of-burn circuits repository is licensed under the MIT License. The Keccak hash function implementation within the circuits is a refactored version of keccak256-circom licensed under GNU GPL v3. The Solidity contracts repository is licensed under the MIT License. All identified components use recognized open source licenses (MIT and GPL v3), making the source code publicly available under proper open source licensing.

Private State Scalability — Infinity grow. WORM uses nullifiers computed as Poseidon2(POSEIDON_NULLIFIER_PREFIX, burnKey) to prevent reuse of burn keys, and these nullifiers are exposed as public inputs in the circuit commitment. The protocol requires on-chain storage of nullifiers to prevent double-spending of burn proofs, and there is no mechanism described for pruning or rotating this nullifier set. Since each proof-of-burn submission adds a new nullifier that must be stored permanently to maintain security, the nullifier set grows indefinitely with protocol usage.

Client-side indexing — No scanning. WORM ERC20 tokens (BETH and WORM) have balances tracked on-chain through the Ethereum state so there is no need for client-side scanning. When users want to mint BETH through private proof-of-burn proofs, they generate the zk-proof on the UI using their recovery file containing their burnKey and receiver address. These process has to

be performed per burn transaction. There is no mechanism to show privately burn balances on the WORM UI so there is no client-side indexing or scanning. Users have to keep track of their burn transaction data on their own.

Private state model — UTXO-based state model. WORM uses a UTXO-based state model. The protocol manages private state through balance commitments called "coins," which are Poseidon3 hashes of a burn key and amount that can be partially withdrawn to create new coins. The system tracks nullifiers (Poseidon2 hashes of burn keys) to prevent double-spending and replay attacks. Each transaction involves proving ownership of a private proof-of-burn and inserting a new nullifier into the on-chain state. This process is similar to standard UTXO models with the caveat that the "coins" are only mintable and their spending does not generate new UTXO outputs.

Private Data Storage — Smart contracts. WORM state is stored in the different smart contracts related to the protocol. The WORM and BETH token data is stored in smart contracts as any other ERC20 token. The nullifier data is stored inside the BETH contract so it is easily accesible when calling the mint function. The burn address and its received funds are stored in Ethereum state, therefore does not directly affect the protocol data storage. The burn key itself, which serves as the user's private key to the protocol, remains off-chain and is never revealed.

Access to DeFi — Unlimited access to DeFi applications. WORM and BETH are both standard ERC-20 tokens deployed on Ethereum mainnet. They can be integrated with any DeFi application that supports ERC-20 tokens, including DEXs, lending protocols, and liquidity pools, without requiring any protocol changes. This provides full composability with the Ethereum DeFi ecosystem. It is worth pointing out that the token utility depends solely on the correct functioning of the WORM protocol. If the protocol experiences a critical failure or vulnerability, the value of WORM and BETH could be severely impacted, which would in turn affect their usability in DeFi applications.

Programmability / Generality — Only payments. WORM's programmability is limited to its core proof-of-burn operations: burning ETH to a derived address, proving the burn via zk-SNARK to mint BETH, and spending BETH with partial withdrawal support through encrypted coin commitments. The circuit enforces a fixed set of constraints around state root verification, nullifier generation, encrypted balance commitments, reveal amounts, and burn-extra commitments for distribution logic. The protocol does not support arbitrary private smart contract logic or custom private computations beyond these predefined burn/mint/spend flows.

zERC20

zERC20 is a fully ERC-20-compliant token with a private-transfer mechanism, usable from a standard wallet. A private transfer sends zERC20 to a cryptographically derived burn address. The recipient later withdraws the same amount to any address by submitting a zero-knowledge proof proving they know the burn secret, so the link between the two addresses is never revealed

on-chain. Wrapper tokens are backed 1:1 by underlying assets such as ETH, USDC, and BNB, and cross-chain private transfers are supported via LayerZero. The token draws from the design of zk-Wormhole / EIP-7503 private proof-of-burn.

Categories: zkWormholes

Documentation: https://zerc20.gitbook.io/zerc20

Confidentiality — No. As a standard ERC-20 token, per-address balances and transfer amounts are publicly visible on every chain it deploys on. The privacy mechanism hides only the link between a burn-address deposit and the recipient's later mint, not the value transferred. Both the burn transfer and the verifier's mint emit cleartext amounts. Unique or unusual amounts can therefore be used to correlate deposit and withdrawal even without the linkage being on-chain.

Anonymity — Unlinkability. Sender and recipient addresses both appear in observable on-chain transactions, but the link between them is broken by the burn-address construction. The burn transaction sending the zERC20 token is publicly visible, but observers cannot tell which withdrawal address the burn address is linked to. Neither party is anonymous in isolation — only the deposit-withdrawal linkage is hidden.

Asset privacy — No. zERC20 deploys a separate ERC-20 contract per wrapped asset (zETH, zUSDC, zBNB), so every transfer is associated with a specific token contract that identifies the underlying asset. No shielded-pool or confidential-asset mechanism hides the token type at the transfer layer.

Plausible deniability — No. Even though Proof of Burn mechanisms can provide plausible deniability for the sender, zERC20 requires users to wrap their tokens (native ETH or some pre-defined ERC20 token) through their smart contract. This interaction is on-chain and visible to any observer, therefore plausible deniability is not achieved. The concept of an "internal private transfer" does not exist in zERC20 because the Private Send feature available in the UI is just a withdraw/teleport action. Senders need to interact with the zERC20 smart contract to send tokens to a burn address and the receivers need to interact with the zERC20 smart contract to mint/withdraw tokens.

Time-to-finality — N/A. zERC20 is an application-layer token, so finality is inherited from each underlying chain rather than introduced by the protocol itself. Per-chain transaction finality is determined by Ethereum, Arbitrum, Base, and BNB Chain respectively. Cross-chain transfers add LayerZero-relayed root aggregation on top, but the local finality of any single chain transaction is the host chain's.

Number of secrets — 1. The user holds only their standard Ethereum wallet private key. zERC20 derives no keys of its own. Each transfer additionally uses a one-time random value the sender samples and shares with the recipient, but this per-transfer value stays ephemeral, never

becoming a backed-up key. Users can download this secret data if they want to. The secret burn data allows the owner to check the burn address and also withdraw funds if they have not been withdrawn yet.

Deposit time — 0. Wrapping into zERC20 is a standard on-chain transaction with no protocol-imposed delay: tokens are obtained by depositing the underlying asset through the frontend or CLI, or by purchasing on a decentralized exchange like Uniswap, all of which complete in a single host-chain transaction. There is no queue, challenge window, or time-lock gating the deposit step.

Withdraw time — 0. Withdrawals can happen in the same chain or in a cross-chain manner. In both cases, the withdrawal process is the same: the recipient submits a zero-knowledge proof to the verifier contract proving they know the burn secret and that a burn transaction was made to the corresponding burn address. According to the FAQ section, private transfers (withdrawals) can take an average time of 30 minutes to 1 hour due to LayerZero messages propagation.

Censorship resistance — No. The zERC20 token contract enforces an OFAC blocklist against a shared per-chain OFAC sanctions registry. This compliance feature prevents blocked addresses from sending, receiving, wrapping, unwrapping and teleporting any zERC20 tokens. Withdrawals submit a zero-knowledge proof directly to the verifier contract, so the relay node is optional rather than required.

External network dependence — Yes, permissionless. Cross-chain private transfers depend on LayerZero for messaging: a hub aggregates transfer roots across chains into a Poseidon tree and broadcasts the global root to per-chain verifiers, adding the messaging network's verifier and executor sets as trust assumptions beyond the underlying chain. An adaptor provides cross-chain exit via Stargate when liquidity is more favourable on another chain. Same-chain transfers do not require this external network.

Escape hatch — No. The zERC20 token contract enforces an OFAC blocklist, and the contract owner can upgrade contracts and rotate verifiers. The blocklist feature prevent blocked addresses from sending, receiving, wrapping, unwrapping and teleporting any zERC20 tokens. There is no escape hatch or ragequit functionality for blocked addresses.

Upgradeability — Single admin. Upgrades are controlled by a contract owner role that can replace contract logic and rotate the proof verifiers, with no on-chain governance or timelock described. The blocklist contract itself is non-upgradeable. The zERC20 token, verifier, hub, and liquidity manager are owner-upgradeable.

Client-side proving — Yes. Withdrawal proofs are generated on the user's device. For single withdrawals the client produces a lightweight Groth16 proof, benchmarked at around 105 ms. For batch withdrawals the client folds the Nova IVC steps locally and a Decider Prover HTTP worker finalises the Nova proof for on-chain verification. The burn secret remains a private witness to the circuit rather than being handed to the decider. The IVC stack is built on Sonobe.

Third-party inspectability — Yes. The off-chain indexer is positioned to link sender activity to recipients. The indexer operator observes the sender, the burn address, and the value, and learns the recipient on query. A relay node submits gasless redeem transactions on behalf of users and swaps zERC20 into native gas tokens, exposing recipient fee authorisations to that operator. The protocol acknowledges this exposure and advises users to run their own indexer instance to avoid sender-recipient linkage leaks.

Implementation maturity — 3 : Mainnet for less than 1 year. Mainnet deployments exist on Ethereum, Arbitrum, Base, and BNB Chain with wrapped tokens zETH, zUSDC, and zBNB. The mainnet zETH contact was deployed on February 4th 2026.

Post-quantum secure — No. The protocol uses Poseidon hashing over the BN254 scalar field with Nova folding and Groth16 for withdrawal proofs, SHA-256 for hash chain commitments, and identity-based encryption for stealth messaging. Pairing-based Groth16 on BN254 rests on elliptic-curve discrete-log and pairing assumptions that Shor's algorithm breaks, and no lattice- or hash-based proof system is used. Wallet-level authorisation remains standard ECDSA, so none of the primitives in use are quantum-resistant.

Layer of enforcement — Asset. Compliance lives in the zERC20 token contract, which is described as enforcing an OFAC blocklist against a shared per-chain OFAC sanctions registry referenced by all zERC20 tokens on the same chain. Enforcement is therefore at the asset (token) layer rather than at an application contract or the chain consensus, with the host chains remaining permissionless.

Enforcement entities — Admin. Compliance enforcement is handled by a designated administrative role rather than a governance process or an external compliance vendor. The trust model assigns a blocklist multi signature owner who can add or remove sanctioned addresses on a shared per-chain OFAC registry.

Type of compliance — Proof of innocence (POI) / ASP. The zERC20 token contract enforces an OFAC blocklist against a shared per-chain OFAC sanctions registry that is non-upgradeable and referenced by all zERC20 tokens on the same chain. This is a curated disallowed-set check enforced on-chain at the token contract. There is an additional Proof of Innocence (POI) mechanism that allow users to prove their received funds did not originate from sanctioned addresses without revealing transaction details. No identity verification, viewing-key disclosure, off-chain attestation oracle, or risk-scoring service is described.

Point of enforcement — Deposit, Transfer, Withdrawal. The zERC20 token contract is described as enforcing an OFAC blocklist while also emitting IndexedTransfer events for transfers and exposing a teleport endpoint for verifier-driven mints, so both the transfer leg and the withdrawal-side mint route through the same enforcing contract. The LiquidityManager contract handles wrap and unwrap of underlying assets, which move zERC20 in and out of user balances.

Selective disclosure: viewing entity — None. No protocol-defined viewer is documented: only Contract Owner, Blocklist Owner, Indexer Operator, and ICP Canisters appear, none of which grant a key-gated read interface to user transaction history. The indexer operator already observes the sender, the burn address, and the value, and learns the recipient on query. The Internet Computer canisters store encrypted data that they cannot decrypt without the recipient's key.

Selective disclosure: viewing control — None. No viewing-key mechanism or permissioned disclosure channel is described. Recipients receive funds via burn-address scanning and a zero-knowledge proof submitted to the verifier, which mints to a destination address — neither flow exposes a handle through which read access could be granted, revoked, or updated. The ICP canisters store encrypted data they cannot decrypt without the recipient's key.

Cryptographic verifiability — Yes. Withdrawals are gated by zero-knowledge proofs verified on-chain: the recipient submits a proof that a zERC20 transfer was made to the corresponding burn address and that they know the secret used to form it, and the verifier mints tokens. Batch withdrawals are folded via Nova IVC, single withdrawals use Groth16, with Merkle membership over a Poseidon tree on BN254. Correctness rests on these proofs rather than attestations or voting, but the verifier is an upgradeable contract — the proving system is swappable by the contract owner.

Open source — No. The repository is published under Business Source License 1.1, with a Change Date of 2030-04-08 and a Change License of GNU General Public License v2.0 or later, which is source-available rather than an OSI-approved license. The Business Source License is explicitly not an Open Source license, and this single license covers the Solidity contracts, the off-chain services, and the CLI in the monorepo. The GPLv2-or-later change does not take effect until the 2030 Change Date, so the codebase is not open source under the OSI definition today.

Private State Scalability — Infinity grow. Protocol-specific data accumulates without bound across several structures. Every transfer appends to the on-chain SHA-256 hash chain, and the indexer maintains a Poseidon Merkle tree it periodically proves on-chain via IVC. Both extend per transfer with no documented pruning. The verifier also persists a per-recipient running total of the amount already withdrawn, keyed forever by recipient. Cross-chain operation also use a Merkle tree — each chain's verifier sends its local root to a hub that builds a global Merkle tree.

Client-side indexing — Always scanning. Users must scan the chain to reconstruct the off-chain Merkle tree from on-chain hash chain commitments. The protocol uses gas-efficient hash chains on-chain and requires off-chain Poseidon-based Merkle tree reconstruction to generate withdrawal proofs. No alternative mechanism for balance tracking without chain scanning is described in the protocol design.

Private state model — Account-based state model. On the base token layer zERC20 is a standard ERC-20 with per-address balances, and the privacy layer tracks the cumulative amount withdrawn per recipient, minting only the difference between the amount received at that recipient and the

amount already withdrawn rather than consuming discrete spend-once notes. Burn-address transfers add recipient-and-value leaves to a Poseidon Merkle tree, but redemption is governed by that running per-recipient total rather than by nullifying individual note commitments.

Private Data Storage — Smart contracts, Off-chain storage with on-chain commitment. Transfer events emit an on-chain SHA-256 hash chain that commits to the per-transfer leaves needed for withdrawal proofs. The Poseidon Merkle tree consumed by proofs is reconstructed off-chain from those events rather than stored on-chain — the hash chain is the gas-efficient commitment, the Merkle tree lives off-chain. A separate stealth-messaging layer on the Internet Computer holds encrypted announcements protected by identity-based encryption for recipient burn-secret discovery.

Access to DeFi — Unlimited access to DeFi applications. zERC20's wrapper tokens are standard ERC-20s, so they are held, traded, lent, or bridged by any wallet, DEX, or lending protocol that already accepts ERC-20s. The design emphasises wallet compatibility through the standard ERC-20 flow, easy dApp integration via existing DEXes and lending protocols, and cross-chain movement via zk-Wormhole. The privacy mechanism activates only when tokens are sent to a burn address, so ordinary holding, trading, and lending behave like any other ERC-20.

Programmability / Generality — Only payments. zERC20 supports private token transfers through the standard ERC-20 interface and can be integrated into existing DEXes and lending protocols. The protocol also enables cross-chain private transfers via a Hub contract. However, the design is limited to payment and transfer operations with no support for arbitrary private smart contract logic or programmable private state beyond basic token movements.

Table of project evaluations

Future Work

Conclusion